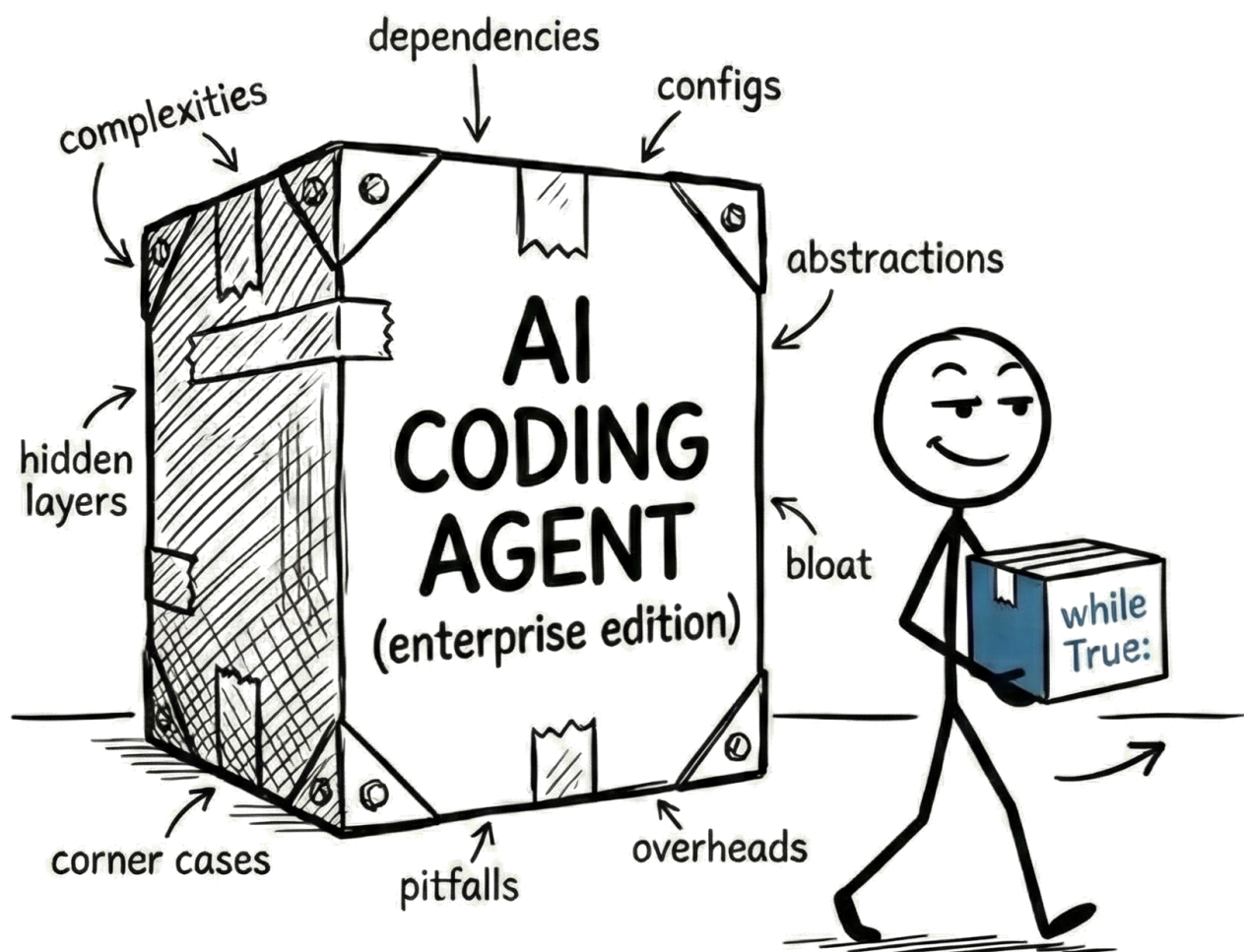


Build Your Own **CODING AGENT**

The Zero-Magic Guide to AI Agents in Pure Python



J. Owen

Build Your Own Coding Agent

The Zero-Magic Guide to AI Agents in Pure Python

J. Owen

This book is available at

<https://leanpub.com/build-your-own-coding-agent>

This version was published on 2026-03-31



© 2026 J. Owen

Tweet This Book!

Please help J. Owen by spreading the word about this book on [Twitter](#)!

The suggested tweet for this book is:

Just built a coding agent from scratch with nothing but HTTP calls and a shell.
Turns out "AI magic" is just a while loop and an API call.

The suggested hashtag for this book is [#buildyourowncodingagent](#).

Find out what other people are saying about the book by clicking on this link to search for this hashtag on Twitter:

[#buildyourowncodingagent](#)

*To my wife, my son, my parents, and my grandparents—you taught me there is
no magic, only effort. Everything I build, I build because of you.*

Contents

Preface	1
Who This Book Is For	1
What You Will Build	1
Testing Approach	1
Code Examples	2
Conventions Used in This Book	3
 Part I: The Brain	 4
 Chapter 1: The Zero Magic Manifesto	 5
What is an Agent, Really?	5
What We Are Building	6
Project Setup	6
The AgentStop Exception	8
The Agent Class	9
Defining Success with Tests	10
The Main Loop	11
Run It	12
Wrapping Up	12
 Chapter 2: The Raw Request	 13
Get an API Key	13
The Vault (.env)	13
The Anatomy of a Request	14
The Code	15
Run It	16
Troubleshooting	17
Cleaning Up	17
Wrapping Up	18
 Chapter 3: The Infinite Loop	 19

CONTENTS

The Illusion of Memory	19
The Testing Problem	19
Response Types	20
The FakeBrain Pattern	21
Defining Success	22
The Claude Class	24
The Agent Class (Updated)	26
The Main Loop (Updated)	27
Verify the Tests Pass	28
Test the Memory	28
The Context Window Problem	29
Wrapping Up	30
Chapter 4: The Universal Adapter	31
The Adapter Pattern	31
HTTP Resilience	31
The Brain Interface	33
The FakeBrain (Updated)	34
The Claude Brain (Refactored)	35
The DeepSeek Brain	36
The BRAINS Registry	37
The Agent Class (Updated)	38
Tests for Multi-Brain Support	40
The Main Loop (Updated)	41
Setting Up DeepSeek	41
Try It	42
“We Just Moved Code Around”	42
Wrapping Up	43
Part II: The Hands	44
Chapter 5: The Tool Protocol	45
How Tools Actually Work	45
Defining the Tool Interface	46
The ReadFile Tool	47
The WriteFile Tool	49
Tool Helpers	50
Updating the Thought Class	51
Updating the Claude Class	52

CONTENTS

The Agent Class with Tools	54
The Main Loop	57
Test It	59
Wrapping Up	60
Chapter 6: The Scratchpad (Memory)	61
The “Zero Magic” Memory	61
The Memory Class	62
The ToolContext Class	63
The SaveMemory Tool	64
Updating the Claude Class	65
Crafting the System Prompt	67
Updating the Agent Class	68
The Main Loop (Updated)	68
Testing Persistence	69
Wrapping Up	70
Chapter 7: The Safety Harness (Plan Mode)	72
The Concept	72
Tests First	73
The WritePlan Tool	74
One List, Two Views	76
Telling the Brain What Mode It’s In	76
The Agent Class (Updated)	77
The Main Loop (Updated)	79
Testing the Harness	80
The Psychology of the “Plan”	81
Wrapping Up	82
Chapter 8: The Context Pipeline (Map & Search)	83
The ListFiles Tool	83
The SearchCodebase Tool	85
Update the Tools List	87
The “Zoom In” Test	88
Wait, is this RAG?	88
Wrapping Up	89
Chapter 9: The Reality Check (Running Code)	91
The Feedback Loop	91
Tests First	92

CONTENTS

The RunCommand Tool	92
The Interactive Trap	94
The Self-Healing Demo	94
The TDD Workflow	95
The Surgical Edit	96
The Closed Loop	98
Hardening the Loop	98
Context Compaction	98
Security Considerations	103
Wrapping Up	104
Part III: The Frontier	105
Chapter 10: Going Dark (Local Models)	106
The Trade-off	106
Installing Ollama	106
The Ollama Brain Class	108
Running with Ollama	111
The “Infinite Loop” Experiment	112
The Practical Differences	112
The Hybrid Workflow	113
Model Selection	114
Troubleshooting Ollama	115
Wrapping Up	115
Chapter 11: The Extension (Web Search)	117
Step 1: The Meta-Prompt	117
Step 2: The Surgery	118
Step 3: The Reference Implementation	118
Step 4: The Tests	120
Self-Modification	121
Wrapping Up	122
Chapter 12: The Capstone (Building a Game)	123
Step 1: Preparation	123
Step 2: The Architect (Plan Mode)	124
Step 3: The Builder (Act Mode)	124
Step 4: The Reality Check	125
Step 5: The Pivot (Feature Creep)	126
What Goes Wrong	126

Wrapping Up	128
Epilogue	128
Appendix A: Streaming Responses	129
How Streaming Works	129
The Implementation	130
What Changed from Chapter 11	133
The Trade-Off	134
Running the Code	134
Acknowledgments	136

Preface

Who This Book Is For

You are a software engineer who is skeptical of AI hype.

You have tried the frameworks—and watched your LangChain app hallucinate its way into deleting a production database. And you think: *“There has to be a better way.”*

There is. This book is for developers who want to understand what is actually happening when an AI agent runs. Not the marketing diagrams. Not the “Reasoning Engine” abstractions. The actual HTTP requests. The actual while loop.

If you can read Python and have built a web app or CLI tool before, you have everything you need.

What You Will Build

Nanocode is a coding agent that runs in your terminal. By the end of this book, it will:

- Read and write files in your codebase
- Execute shell commands
- Search code using pure Python
- Remember context across sessions
- Separate planning from execution with a safety mode
- Search the web for documentation and answers

You will build it from scratch using `requests`, `python-dotenv`, and `pytest`; shell access comes from Python’s standard-library `subprocess` module. No LangChain. No vector databases. No “orchestration frameworks.” Just Python you can debug with `print()`.

The one exception: Chapter 11 adds `ddgs` for web search—a single lightweight dependency.

Testing Approach

This book uses a “test-alongside” approach. For most features:

1. We introduce the concept—why we need this
2. We show the test first, so you know what success looks like
3. We implement the code to make the test pass
4. We verify with pytest

This teaches *Test-Driven Development* thinking without the full red-green-refactor ceremony in print. It also solves a critical problem: you cannot test an LLM-powered application by actually calling the LLM. API calls are slow, expensive, and non-deterministic.

Starting in Chapter 3, we introduce the FakeBrain pattern—a test double that returns predictable responses. This lets you run your entire test suite without making a single API call or spending a single cent.

Code Examples

This book follows a “Code First” approach. Each chapter builds on the previous one, and every code example is extracted from working files.

To get the code:

- **GitHub:** Clone or download from GitHub.¹
- **Leanpub:** The complete source code is also included in the downloadable resources with your purchase.

The code is organized by chapter (ch01/, ch03/, ch04/, etc.) plus an appendix/ folder. Most folders contain:

- `nanocode.py` – The complete, runnable agent for that chapter
- `test_nanocode.py` – Tests that verify the code works without API calls

¹<https://github.com/optimalone/build-your-own-coding-agent>

Two exceptions: `ch02/` contains only a standalone `test_api.py` script (used once and discarded), and `ch12/`'s agent snapshot is functionally the same as `ch11/`, with added capstone artifacts.

You can copy any chapter folder and pick up from there. Run `pytest` to verify your code matches the expected behavior.

Conventions Used in This Book

Throughout the book, you will see three types of callouts:

Dev Tip: Architectural wisdom that applies beyond this project. These are patterns you can reuse in your own work.



Warning: Security or safety risks. Pay attention to these—ignoring them can lead to deleted files or leaked API keys.



Aside: Deep dives and tangents. Useful context, but you can skip them on a first read without losing the thread.

Code walkthroughs follow a consistent pattern: first the context (why we need this code), then the code itself, then a line-by-line explanation of the important parts.

Time to write some code.

Part I: The Brain

Part I builds the skeleton of an agent: a `while` loop that talks to Claude via raw HTTP. By Chapter 4, you'll support multiple LLM providers through the Adapter Pattern—and understand that an “AI agent” is just a loop, a brain, and a list of messages.

Chapter 1: The Zero Magic Manifesto

If you have tried to build an AI application in the last two years, you have likely felt “Framework Fatigue.”

You install a popular library. You import a `ReasoningEngine`. You call `.run()`. It works like magic for the “Hello World” example. But the moment you try to do something real—like editing a specific line in a Python file without deleting the imports—it breaks.

And because you used a framework, you can’t fix it. You are stuck digging through layers of abstract classes, factory patterns, and “Chains” trying to find the one prompt that is causing the hallucination.

We are not going to do that here.

This book is a rebellion against “Magic.” We are going to take the “Zero Magic” approach: build a production-grade coding agent called **Nanocode** in pure Python. No LangChain, no AutoGPT, no Pydantic.

Why? Because an autonomous agent is not magic. It is just a `while` loop.

What is an Agent, Really?

Strip away the venture capital marketing, and an “Agent” is just a **thermostat**.

A thermostat reads the temperature (input), compares it to the target (decision), and turns on the heater (action). Then it waits and repeats. That’s it. An AI agent does the same thing, just with text instead of temperature.

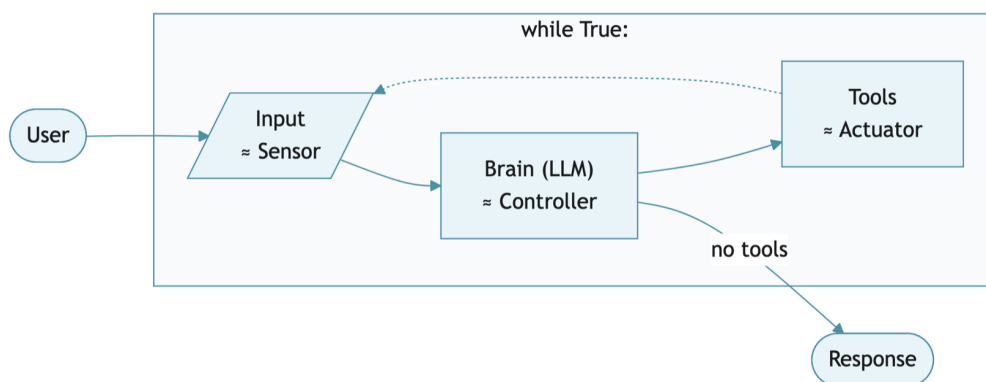


Figure 1. The Agent Loop: User input flows through a while loop where Input (Sensor) feeds the Brain/LLM (Controller), which triggers Tools (Actuator), cycling back to Input until the Brain outputs a Response.

More specifically, an agent has four parts. The *Brain* is the LLM—a stateless function where you send text and it returns text. It calls *Tools*—functions like Read File and Run Command—to interact with the outside world. All of this sits inside a *Loop* (a `while True`) that keeps cycling until the task is done, with *Memory*—just a Python list—accumulating the conversation history along the way. (The list dies when the program ends; we’ll add persistent storage in Chapter 6.)

If you can write a while loop, you can build an agent.

By building it from scratch, you will have something the framework users don’t control. When our agent gets stuck in a loop, you will know exactly which line of code caused it. When the API bill gets too high, you will see exactly where the tokens are leaking.

What We Are Building

Nanocode is a CLI tool that runs in your terminal. You talk to it like a colleague. It reads your files. It runs your commands. It edits your code.

By the end of this book, you will have wired it up to Claude Sonnet 4.6 (or DeepSeek, or a local model via Ollama). You’ll give it hands—tools to read files, write files, and run shell commands—and eyes to search your codebase. And you’ll build a safety harness so it can’t accidentally `rm -rf /`.

Project Setup

Dev Tip: The biggest threat to an AI project is “Dependency Rot.” AI libraries move fast and break things. By sticking to requests and subprocess, this agent will likely still run 5 years from now.

1. Initialize the Project

```
1 mkdir nanocode
2 cd nanocode
3 git init
```

2. Create a Virtual Environment

Never install AI tools globally. They conflict with system packages.

```
1 # Mac/Linux
2 python3 -m venv venv
3 source venv/bin/activate
4
5 # Windows
6 python -m venv venv
7 venv\Scripts\activate
```

3. Install Dependencies

We only need three libraries:

- requests – To talk to the LLM APIs.
- python-dotenv – To load API keys from a .env file.
- pytest – To test our code without making API calls.

Create requirements.txt:

```
1 requests
2 python-dotenv
3 pytest
```

Install:

```
1 pip install -r requirements.txt
```

4. Secure Your Keys



Warning: If you push your API key to GitHub, bots will scrape it and drain your account within minutes.

Create `.gitignore`:

```
1 .env
2 __pycache__/
3 venv/
4 .DS_Store
5 .nanocode/
```

The AgentStop Exception

Before we write the event loop, we need a clean exit mechanism. An exception beats break statements scattered across the code.

The Context: Exceptions are not just for errors; they are also a control flow mechanism. When the user types `/q`, we raise `AgentStop`. The main loop catches it and exits cleanly.

The Code:

```

1  # --- Exceptions ---
2
3  class AgentStop(Exception):
4      """Raised when the agent should stop processing."""
5      pass

```

This goes at the top of `nanocode.py`. It's a marker exception—no logic, just a signal.

The Agent Class

Now the core abstraction: the `Agent` class. It holds state and logic in one place, which makes it easy to test.

The Context: We could put all the logic in `main()`. But then we'd have to mock `input()` and `print()` to test it. By extracting the logic into `Agent.handle_input()`, we can test it directly.

The Code:

```

10 class Agent:
11     """A coding agent that processes user input."""
12
13     def __init__(self):
14         pass
15
16     def handle_input(self, user_input):
17         """Handle user input. Returns output string, raises AgentStop to
18         ↔ quit."""
19         if user_input.strip() == "/q":
20             raise AgentStop()
21
22         if not user_input.strip():
23             return ""
24
25         return f"You said: {user_input}\n(Agent not yet connected)"

```

The Walkthrough:

- **Lines 13-14:** Empty constructor for now. We'll add `brain` and `tools` in later chapters.

- **Lines 18-19:** The `/q` command raises `AgentStop` instead of returning a special value—the caller decides how to handle shutdown.
- **Line 24:** Echo the input back. This is a placeholder—later, we'll send this to the Brain.

Defining Success with Tests

Before we write the main loop, we need tests.

Create `test_nanocode.py`:

```

1  import pytest
2  from nanocode import Agent, AgentStop
3
4
5  def test_handle_input_returns_string():
6      """Verify handle_input returns a string for normal input."""
7      agent = Agent()
8      result = agent.handle_input("hello")
9      assert isinstance(result, str)
10     assert "hello" in result
11
12
13  def test_empty_input_returns_empty_string():
14      """Verify empty/whitespace input returns empty string."""
15      agent = Agent()
16      assert agent.handle_input("") == ""
17      assert agent.handle_input(" ") == ""
18      assert agent.handle_input("\n") == ""
19
20
21  def test_quit_command_raises_agent_stop():
22      """Verify /q raises AgentStop exception."""
23      agent = Agent()
24      with pytest.raises(AgentStop):
25          agent.handle_input("/q")
26
27
28  def test_quit_command_with_whitespace():
29      """Verify /q works with surrounding whitespace."""
30      agent = Agent()
31      with pytest.raises(AgentStop):
32          agent.handle_input(" /q ")

```

Run the tests:

```
1 pytest test_nanocode.py -v
```

```
1 test_nanocode.py::test_handle_input_returns_string PASSED
2 test_nanocode.py::test_empty_input_returns_empty_string PASSED
3 test_nanocode.py::test_quit_command_raises_agent_stop PASSED
4 test_nanocode.py::test_quit_command_with_whitespace PASSED
```

All green. Our agent handles the basic cases correctly.



Aside: Why pytest? It discovers functions starting with `test_` and runs them. No boilerplate, no classes required. The test code itself is plain Python—no magic.

The Main Loop

Now the thin I/O wrapper that connects the agent to the terminal:

```
29 def main():
30     agent = Agent()
31     print("⚡ Nanocode v0.1 initialized.")
32     print("Type '/q' to quit.")
33
34     while True:
35         try:
36             user_input = input("\n> ")
37             output = agent.handle_input(user_input)
38             if output:
39                 print(output)
40
41         except (AgentStop, KeyboardInterrupt):
42             print("\nExiting...")
43             break
44
45 if __name__ == "__main__":
46     main()
47
```

The Walkthrough:

- **Lines 30–32:** Create the agent and print startup messages.
- **Line 36:** `input()` blocks and waits for the user to type something.
- **Lines 37–39:** Call `handle_input()` and print any output.
- **Lines 41–43:** Catch `AgentStop` (from `/q`) or `KeyboardInterrupt` (from `Ctrl+C`) and break out of the loop.



Aside: Python's `input()` reads one line at a time. All prompts in this book are single-line. This keeps the code simple—production agents use richer input methods like `readline` or full TUIs.

Notice the separation: `Agent.handle_input()` contains all the logic. `main()` is just I/O glue. This makes the agent testable without mocking `stdin/stdout`.

Run It

```
1 python nanocode.py
```

You should see:

```
1 ⚡ Nanocode v0.1 initialized.
2 Type '/q' to quit.
3
4 > hello
5 You said: hello
6 (Agent not yet connected)
7
8 > /q
9
10 Exiting...
```

This is the chassis. The engine comes next.

Wrapping Up

That's the chassis: an `Agent` class, a `handle_input()` method, a `while True` loop. It doesn't do anything useful yet—but everything we build from here plugs into this skeleton. The tests make sure we don't break what already works as we go.

Chapter 2: The Raw Request

Most tutorials will tell you to `pip install anthropic`. We are not going to do that.

SDKs hide the truth. They add layers of abstraction that make “Hello World” easy but make debugging “Error 400” a nightmare. When you learn the SDK, you learn the SDK. When you learn the raw HTTP call, you learn the protocol underneath every SDK.

We are going to send a message to Claude using nothing but the `requests` library. Claude is Anthropic’s flagship LLM—one of the most capable models for coding tasks.

Get an API Key

To talk to Claude, you need an API Key. This is a long string of characters that acts like a password tied to your billing account.

1. Go to the Anthropic Console.¹
2. Sign up and add a payment method (minimum \$5 credit).
3. Create a new API Key and name it `nanocode`.
4. Copy the key (it starts with `sk-ant-...`).



Warning: Treat this key like a password. Anyone with it can spend your money.

The Vault (.env)

We need a safe place to store this key. We never put keys directly in code. Create a file named `.env` in your project root:

¹<https://console.anthropic.com/settings/keys>

```
1 touch .env
```

Open it and paste your key:

```
1 ANTHROPIC_API_KEY=sk-ant-api03-...
```

We installed `python-dotenv` in Chapter 1 for exactly this purpose—it reads `.env` and loads the values into `os.environ`.

The Anatomy of a Request

To talk to an LLM, we send an HTTP POST request to:

`https://api.anthropic.com/v1/messages`

This request needs three things: authentication in the headers (your API key), configuration in the body (which model, how many tokens), and the message itself.

The Headers

Anthropic requires three headers:

Header	Value	Purpose
<code>x-api-key</code>	Your secret key	Authentication
<code>anthropic-version</code>	<code>2023-06-01</code>	API version
<code>content-type</code>	<code>application/json</code>	Format

The Payload

The “Messages API” expects a list of message dictionaries:

```
1 "messages": [  
2     {"role": "user", "content": "Hello, world!"}  
3 ]
```

Each message has a role (either "user" or "assistant") and content (the text).

The Code

Create a file called `test_api.py`. This is a “smoke test” to prove our connection works. We will delete it later.

The Context: We are writing linear, procedural code. No functions, no classes. We want to see the bare metal.

The Code:

```
1 import os  
2 import requests  
3 import json  
4 from dotenv import load_dotenv  
5  
6 # 1. Load the vault  
7 load_dotenv()  
8 api_key = os.getenv("ANTHROPIC_API_KEY")  
9  
10 # Basic check so we don't crash with a confusing "NoneType" error later  
11 if not api_key:  
12     print("Error: ANTHROPIC_API_KEY not found in .env")  
13     exit(1)  
14  
15 # 2. Define the target  
16 url = "https://api.anthropic.com/v1/messages"  
17  
18 # 3. Authenticate  
19 headers = {  
20     "x-api-key": api_key,  
21     "anthropic-version": "2023-06-01",  
22     "content-type": "application/json"  
23 }  
24  
25 # 4. Construct the payload  
26 payload = {  
27     "model": "claude-sonnet-4-6",  
28     "max_tokens": 4096,  
29     "messages": [  
30         {"role": "user", "content": "Hello, world!"},  
31         {"role": "assistant", "content": "Hello, user!"},  
32     ]  
33 }
```

```

30         {"role": "user", "content": "Hello, are you ready to code?"}
31     ]
32 }
33
34 # 5. Fire! (No safety net)
35 print("🔥 Sending request to Claude...")
36 response = requests.post(url, headers=headers, json=payload, timeout=120)
37
38 # 6. Inspect the raw result
39 print(f"Status: {response.status_code}")
40
41 if response.status_code == 200:
42     # Success: Print the beautiful JSON
43     print("Response:")
44     print(json.dumps(response.json(), indent=2))
45 else:
46     # Failure: Print the ugly raw text so we can debug
47     print("Error:", response.text)

```

The Walkthrough:

- **Line 7:** `load_dotenv()` finds the `.env` file and loads variables into `os.environ`.
- **Line 8:** We grab the API key. Never hardcode this.
- **Lines 11-13:** Basic sanity check. Without this, a missing key causes a confusing `NoneType` error in the headers dict.
- **Line 21:** The `anthropic-version` header is mandatory. Omit it, and the API rejects you.
- **Line 27:** `claude-sonnet-4-6` specifies which model we want.
- **Line 28:** `max_tokens` is required. It caps the response length and prevents runaway costs.
- **Line 36:** We fire the request with a 2-minute timeout. No `try/except`—if the network is down, let Python crash. You need to see where it fails.
- **Lines 41-47:** Check the status code. 200 means success (pretty-print the JSON). Anything else means we print the raw error text for debugging.

Run It

```
1 python test_api.py
```

If everything works, you should see:

```
Status: 200
Response:
{
  "id": "msg_01...",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Hello! Yes, I'm ready to code..."
    }
  ],
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 15,
    "output_tokens": 81
  }
}
```

Troubleshooting

Error	Cause	Fix
401 Unauthorized	Bad API key	Check .env is loading. Print <code>os.environ.get("ANTHROPIC_API_KEY")</code> to verify.
400 Bad Request	Malformed JSON	Did you forget <code>max_tokens</code> ? Is <code>messages</code> a list?
429 Rate Limit	Too many requests or no credits	Wait, or add credits to your account.

Cleaning Up

We proved we can talk to the brain. Delete `test_api.py`—we’re done with it.



Aside: This `test_api.py` is throwaway code—a one-time smoke test. Real automated tests (with `FakeBrain` and `pytest`) come in Chapter 3. You should always delete this file after verifying your API connection works.

Dev Tip: Every API call costs money. Input tokens (what you send) are cheap. Output tokens (what the model writes) are ~5x more expensive. Keep your prompts concise.



Aside: To monitor your spending, check the Usage tab in the Anthropic Console. As of early 2026, a typical coding session with 20–30 exchanges costs \$0.10–\$0.50 with Claude Sonnet. The `usage` field at the bottom of the response JSON shows exact token counts—you could log these to track costs programmatically.

Wrapping Up

That’s a raw API call: headers, JSON payload, response parsing. No abstraction between you and the wire. When something breaks (and it will), you’ll know exactly which layer failed because there’s only one layer.

One problem: Claude has total amnesia. Every request is a blank slate. We’re going to fake memory by replaying the entire conversation history on every turn.

Chapter 3: The Infinite Loop

We have a problem.

Imagine running the script from Chapter 2 twice. Say “My name is Alice.” Claude says hello. Run it again and ask “What is my name?” Claude says “I don’t know.”

This is because LLMs are *stateless*. They have total amnesia. Every request is the first time you have ever met.

To build an agent, we need to fix this by creating artificial memory.

The Illusion of Memory

“Memory” in an LLM isn’t a hard drive. It’s a log file.

When you chat with ChatGPT, it doesn’t “remember” what you said 5 minutes ago. Behind the scenes, the code sends the **entire conversation history** back to the model with every new message.



Figure 2. Context accumulation: Turn 1 sends just “User: Hi” to the API. Turn 2 sends the full history—“User: Hi”, “Assistant: Hello”, “User: How are you?”—to the API.

The model sees the full transcript every time. That’s the trick.

Let’s implement this *context loop* manually. But first, we need to make our code testable.

The Testing Problem

Here’s a hard truth: you cannot test an LLM-powered application by actually calling the LLM.

API calls are slow (2-10 seconds each), expensive (real money per call), and non-deterministic (you might get a different response every time). Imagine running a test suite that costs \$5 and takes 20 minutes. You'd never run it.

The solution is *dependency injection*. Instead of hardcoding the API call inside our agent, we pass in a “brain” object. In production, the brain is Claude. In tests, the brain is a fake that returns predictable responses.

We'll establish this pattern now, before writing any more production code.

Response Types

Before we build the brain, we need to define what it returns. Claude's API sends back complex JSON with multiple content blocks. We need simple Python objects to work with.

The Context: Claude can return text, tool calls, or both in a single response. We need plain data objects to represent these possibilities. (We skip `@dataclass` deliberately—these classes are simple enough that the decorator would save a few lines while hiding what `__init__` actually does.)

The Code:

```
17 class ToolCall:
18     """A tool invocation request from the brain."""
19
20     def __init__(self, id, name, args):
21         self.id = id
22         self.name = name
23         self.args = args # dict
```

`ToolCall` represents the brain asking us to execute a tool. The `id` is a unique identifier for tracking (Claude needs it when we report results back). The `name` is which tool to run. The `args` is a dictionary of parameters.

We won't use `ToolCall` yet—the brain can't call tools—but we define it now because it's part of the `Thought` response type. When we add tools, Claude will return these when it wants to read a file or execute a command.

```

26 class Thought:
27     """Standardized response from any Brain."""
28
29     def __init__(self, text=None, tool_calls=None, thinking=None):
30         self.text = text # str or None
31         self.tool_calls = tool_calls or [] # list of ToolCall
32         self.thinking = thinking # str or None

```

A Thought is what the brain returns after thinking. It might have text, tool calls, both, or neither. The thinking field captures the model's reasoning summary—we'll see where that comes from when we build the Claude class below. This abstraction will let us swap Claude for DeepSeek later without changing any other code.

The FakeBrain Pattern

Now we can build a fake brain for testing.

The Context: We need a brain that returns predictable responses, tracks how many times it was called, and records what conversation it received.

The Code:

```

class FakeBrain:
    """Fake brain for testing - returns predictable responses."""

    def __init__(self, responses=None):
        self.responses = responses or [Thought(text="Fake response")]
        self.call_count = 0
        self.last_conversation = None

    def think(self, conversation):
        self.last_conversation = list(conversation) # Store a copy
        if self.call_count < len(self.responses):
            response = self.responses[self.call_count]
            self.call_count += 1
            return response
        return Thought(text="No more responses")

```

This goes in `test_nanocode.py`, not the production code. Notice that FakeBrain has the same interface as our real brain will—a `think()` method that takes a conversation and returns a Thought.



Aside: This pattern—replacing a real dependency with a predictable fake for testing—is called *test doubling*. Martin Fowler’s article “Mocks Aren’t Stubs”¹ explains the variations (fakes, stubs, mocks, spies). For LLM testing, a simple fake with canned responses is usually all you need.

Defining Success

Before writing the production code, let’s define what success looks like. These tests will guide our implementation.

Test 1: The brain returns a response

```
1 def test_handle_input_returns_brain_response():
2     """Verify handle_input returns the brain's response text."""
3     brain = FakeBrain(responses=[Thought(text="Hello from brain!")])
4     agent = Agent(brain=brain)
5     result = agent.handle_input("hi")
6     assert result == "Hello from brain!"
```

Notice we pass `brain=brain` to the `Agent`. This is dependency injection in action.

Test 2: Conversation accumulates

```
1 def test_conversation_accumulates():
2     """Verify conversation list grows with each interaction."""
3     brain = FakeBrain(responses=[
4         Thought(text="Response 1"),
5         Thought(text="Response 2")
6     ])
7     agent = Agent(brain=brain)
8
9     agent.handle_input("First message")
10    assert len(agent.conversation) == 2 # user + assistant
11
12    agent.handle_input("Second message")
13    assert len(agent.conversation) == 4 # 2 users + 2 assistants
```

¹<https://martinfowler.com/articles/mocksArentStubs.html>

After each exchange, the conversation should have both the user message and the assistant response.

Test 3: Correct message structure

```

1 def test_conversation_contains_correct_roles():
2     """Verify conversation has correct role alternation."""
3     brain = FakeBrain(responses=[Thought(text="AI response")])
4     agent = Agent(brain=brain)
5
6     agent.handle_input("User message")
7
8     assert agent.conversation[0]["role"] == "user"
9     assert agent.conversation[0]["content"] == "User message"
10    assert agent.conversation[1]["role"] == "assistant"
11    assert agent.conversation[1]["content"] == "AI response"

```

The messages must have the exact format Claude expects: {"role": "user", "content": "..."}.

Test 4: Brain receives the conversation

```

1 def test_brain_receives_conversation():
2     """Verify brain.think is called with the conversation list."""
3     brain = FakeBrain()
4     agent = Agent(brain=brain)
5
6     agent.handle_input("Test message")
7
8     assert brain.last_conversation is not None
9     assert len(brain.last_conversation) == 1
10    assert brain.last_conversation[0]["content"] == "Test message"

```

The brain must receive the full conversation, not just the current message.

Run these tests now—they should all fail:

```

1 pytest test_nanocode.py -v

```

```

1 FAILED test_nanocode.py::test_handle_input_returns_brain_response
2 FAILED test_nanocode.py::test_conversation_accumulates
3 ...

```

Good. Now let's make them pass.

The Claude Class

Now the real brain.

The Context: We need a class that wraps the Claude API. It should handle authentication, send conversation history, and parse the response into a Thought. We also enable *extended thinking*—a feature where the model writes internal scratch-work before answering. Think of it as the model talking to itself on a notepad before speaking. It costs extra tokens, but the quality improvement is significant, especially once we add tools in Chapter 5 where the model needs to reason about which tool to call and why.

The Code:

```

37 class Claude:
38     """Claude API - the brain of our agent."""
39
40     def __init__(self):
41         self.api_key = os.getenv("ANTHROPIC_API_KEY")
42         if not self.api_key:
43             raise ValueError("ANTHROPIC_API_KEY not found in .env")
44         self.model = "claude-sonnet-4-6"
45         self.url = "https://api.anthropic.com/v1/messages"
46
47     def think(self, conversation):
48         headers = {
49             "x-api-key": self.api_key,
50             "anthropic-version": "2023-06-01",
51             "content-type": "application/json"
52         }
53         payload = {
54             "model": self.model,
55             "max_tokens": 16000,
56             "thinking": {
57                 "type": "enabled",
58                 "budget_tokens": 10000
59             },
60             "messages": conversation

```

```

61         }
62
63         response = requests.post(self.url, headers=headers, json=payload,
64                                 ↪ timeout=120)
65         response.raise_for_status()
66         return self._parse_response(response.json()["content"])

```

The Walkthrough:

- **Lines 41-43:** Load the API key and fail fast if it's missing.
- **Lines 44-45:** Store config. We'll make the model configurable later.
- **Line 47:** The `think()` method is the brain's interface—same as `FakeBrain`.
- **Lines 55-59:** We enable *extended thinking*—the model produces a reasoning summary before responding, improving quality on complex tasks at the cost of more tokens. `budget_tokens` caps how many tokens the model can spend reasoning (10,000 here)—those tokens count toward your bill just like output tokens. In our setup, `max_tokens` covers the total output including both thinking and the response, so Anthropic requires it to exceed `budget_tokens`. With 10,000 thinking tokens and 16,000 max, the response itself can use up to 6,000 tokens.
- **Line 60:** The payload includes `"messages": conversation`—the full history, not just the current message. This is the context loop.
- **Line 65:** Parse Claude's complex response format into our simple `Thought`.

Now the response parser:

```

67 def _parse_response(self, content):
68     """Convert Claude's response format to Thought."""
69     text_parts = []
70     tool_calls = []
71     thinking = None
72
73     for block in content:
74         if block["type"] == "thinking":
75             thinking = block["thinking"]
76         elif block["type"] == "text":
77             text_parts.append(block["text"])
78         elif block["type"] == "tool_use":
79             tool_calls.append(ToolCall(
80                 id=block["id"],
81                 name=block["name"],
82                 args=block["input"]

```

```

83         ))
84
85     return Thought(
86         text="\n".join(text_parts) if text_parts else None,
87         tool_calls=tool_calls,
88         thinking=thinking
89     )

```

Claude’s API returns a list of “content blocks.” Each block has a type—“thinking”, “text”, or “tool_use”. The thinking block arrives first, containing a summary of the model’s reasoning—we store it on the Thought for the caller to display. Text blocks become the response, and tool_use blocks become ToolCall objects. The parser doesn’t print anything; it just converts the raw JSON into a clean Thought.

The Agent Class (Updated)

Now we update the Agent from Chapter 1 to accept a brain and maintain conversation history.

The Code:

```

94 class Agent:
95     """A coding agent with conversation memory."""
96
97     def __init__(self, brain):
98         self.brain = brain
99         self.conversation = []
100
101     def handle_input(self, user_input):
102         """Handle user input. Returns output string, raises AgentStop to
103         ↪ quit."""
104         if user_input.strip() == "/q":
105             raise AgentStop()
106
107         if not user_input.strip():
108             return ""
109
110         self.conversation.append({"role": "user", "content": user_input})
111
112     try:
113         thought = self.brain.think(self.conversation)
114         if thought.thinking:
115             lines = thought.thinking.strip().split("\n")[:5]

```

```

115         for i, line in enumerate(lines):
116             prefix = "  " if i == 0 else "    "
117             print(f"\033[2m{prefix}{line}\033[0m")
118         text = thought.text or ""
119         self.conversation.append({"role": "assistant", "content": text})
120         return text
121     except Exception as e:
122         self.conversation.pop() # Remove failed user message
123         return f"Error: {e}"

```

The Walkthrough:

- **Lines 97-99:** Accept a brain via dependency injection. Initialize an empty conversation list.
- **Line 109:** Append the user's message to history *before* calling the brain.
- **Lines 112-120:** Call the brain, display up to five lines of thinking in dimmed text (`\033[2m` is the ANSI escape code for dim, `\033[0m` resets it), extract the response, and append it to history.
- **Lines 121-123:** If the API call fails, remove the user message we just added. This keeps the conversation in a valid state.

Pay attention to line 109: we add the user message *before* calling the brain. The brain needs to see the full conversation including the current message.

Dev Tip: When things go wrong, your first debugging tool is `print(self.conversation)`. This shows exactly what the brain is seeing. Malformed messages, missing roles, or truncated content become obvious when you inspect the raw list.

The Main Loop (Updated)

The main loop is now just a thin I/O wrapper:

```

128 def main():
129     brain = Claude()
130     agent = Agent(brain)
131     print("⚡ Nanocode v0.2 (Conversation Memory)")
132     print("Type '/q' to quit.\n")
133
134     while True:
135         try:
136             user_input = input("> ")
137             output = agent.handle_input(user_input)
138             if output:
139                 print(f"\n{output}\n")
140
141         except (AgentStop, KeyboardInterrupt):
142             print("\nExiting...")
143             break
144
145 if __name__ == "__main__":
146     main()
147

```

All the logic is in the Agent class. The loop just reads input, calls `handle_input()`, and prints the result. This separation makes the agent testable—we test `Agent.handle_input()` directly without needing to mock `input()` or `print()`.

Verify the Tests Pass

Run the tests again:

```

1  pytest test_nanocode.py -v

1  test_nanocode.py::test_handle_input_returns_brain_response PASSED
2  test_nanocode.py::test_conversation_accumulates PASSED
3  test_nanocode.py::test_conversation_contains_correct_roles PASSED
4  test_nanocode.py::test_brain_receives_conversation PASSED

```

All green. The tests verify our implementation without making a single API call.

Test the Memory

Now test with the real brain:

```
1 python nanocode.py
```

Try this conversation:

```
1 > I am building a Python agent.
2   🗨 The user is telling me about their project. They want to build
3     a Python agent. I should respond helpfully and ask what kind
4     of agent they're building.
5
6 That sounds exciting! What kind of agent are you building?
7
8 > What language am I using?
9   🗨 The user previously said they are building a Python agent.
10    The answer is Python.
11
12 You are using Python.
```

The conversation list is doing its job.

The Context Window Problem

You might be thinking: “Can I keep this running forever?”

No.

Every loop iteration, the messages list grows:

Turn	Approximate Tokens
1	50
10	5,000
100	50,000

Eventually, you hit the *context limit*—200k tokens for Claude Sonnet, 128k for DeepSeek, as low as 4k for some local models. Exceed it, and the API returns 400 Bad Request. Our error handling on line 121 catches this and reports the error, so the agent won’t crash silently. But the conversation is effectively stuck—every subsequent message will fail too, since the history is still too long.

For now, restarting the agent clears the history and gets you back on track. We'll add proper *context compaction*—tracking token usage from the API response and automatically summarizing old messages before they overflow—when we build the feedback loop in Chapter 9. That's where conversations actually blow up, and where the fix will earn its keep.

Dev Tip: Network calls take 2-10 seconds. The thinking display on lines 113-117 gives users immediate visual feedback that the agent is working. Without it, they'd stare at a frozen prompt and reach for Ctrl+C.

Wrapping Up

Claude remembers now—or rather, we've tricked it into thinking it does. The conversation list grows with every turn, and FakeBrain lets us test the whole thing without spending a cent.

Both patterns will carry through the rest of the book. Every brain we build (Claude, DeepSeek, Ollama) will implement the same `think()` interface, and FakeBrain will test them all.

One loose end: our code is hardwired to Anthropic's API. If we want to add DeepSeek or a local model, we'd have to duplicate a lot of code.

Chapter 4: The Universal Adapter

Look at your Claude class from Chapter 3. It works, but it's limited.

If we want to test without API calls, we need a fake that looks like Claude—but there's no common interface. And if we later want to switch from Claude to DeepSeek, we'd have to create a new class from scratch.

Adding a new provider shouldn't require rewriting existing code. Right now it would.

The fix is the *Adapter Pattern*.

The Adapter Pattern

The Adapter Pattern is just a translation layer.

Think of a power adapter for international travel. Your laptop doesn't know if it's plugged into a US, UK, or EU outlet. The adapter handles the translation. Your code should work the same way.

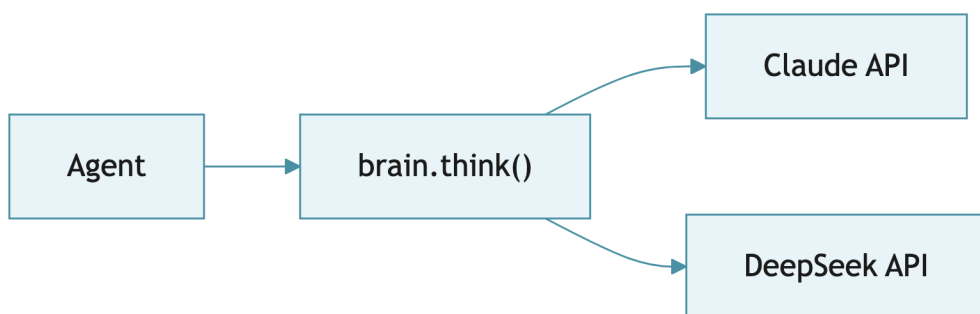


Figure 3. The Adapter Pattern: Agent calls `brain.think()` (the interface), which talks to different providers. The interface is the only contract—implementations can vary.

Our agent shouldn't know or care that we're using Anthropic. It should just know how to "ask the brain."

We're going to define a Brain interface and make every provider implement it.

HTTP Resilience

Before we build the Brain interface, let's add a helper for API resilience. API calls can fail due to rate limits (HTTP 429) or server errors (5xx). Instead of crashing, we'll retry with exponential backoff:

```

11 def request_with_retry(url, headers, payload, max_retries=10):
12     """Make HTTP POST with retry on rate limit (429), server errors (5xx), and
13     ↳ network failures."""
14     for attempt in range(max_retries):
15         try:
16             response = requests.post(url, headers=headers, json=payload,
17             ↳ timeout=120)
18         except requests.exceptions.RequestException as e:
19             wait_time = 2 ** attempt
20             print(f"Network error: {e}. Retrying in {wait_time}s...")
21             time.sleep(wait_time)
22             continue
23
24         if response.status_code == 429 or response.status_code >= 500:
25             retry_after = response.headers.get("retry-after")
26             try:
27                 wait_time = int(retry_after) if retry_after else 2 ** attempt
28             except (ValueError, TypeError):
29                 wait_time = 2 ** attempt
30             print(f"Error {response.status_code}. Retrying in {wait_time}s...")
31             time.sleep(wait_time)
32             continue
33
34         if response.status_code >= 400:
35             try:
36                 error_msg = response.json()["error"]["message"]
37             except (KeyError, ValueError):
38                 error_msg = response.text
39             raise Exception(f"API error ({response.status_code}): {error_msg}")
40
41         return response
42
43     raise Exception(f"Request failed after {max_retries} retries")

```

The Walkthrough:

- **Lines 13-15:** Loop through retry attempts. Wrap the request in try/except to catch network-level failures (connection errors, timeouts, DNS failures).

- **Lines 16-20:** If network fails, wait with exponential backoff and retry.
- **Lines 22-29:** If we get a 429 (rate limited) or 5xx (server error), check the `retry-after` header first – APIs send this to tell you exactly how long to wait. The header can be an integer (seconds) or an HTTP-date string, so we wrap the parse in try/except and fall back to exponential backoff if it's not a simple integer.
- **Lines 32-37:** For other errors (4xx), extract the actual error message from the API response body instead of a generic “400 Client Error.” This surfaces messages like “prompt is too long: 207837 tokens > 200000 maximum.”
- **Line 39:** If successful, return the response.
- **Line 41:** If all retries fail, raise an exception.



Warning: Add `import time` at the top of `nanocode.py`—the `time.sleep()` call needs it.

All Brain classes will use this helper instead of calling `requests.post` directly.

Dev Tip: This retry logic becomes critical in longer agentic loops. When your agent makes 20 API calls while completing a task, a single transient 429 or 503 shouldn't abort the entire operation. The `request_with_retry` helper keeps the loop running through temporary failures.

Dev Tip: Our backoff is deterministic—two agents hitting the same error at the same time will retry at the same time, creating a “thundering herd” that hammers the API in synchronized waves. Production agents add *jitter*: something like `wait_time *= (1 + random.random() * 0.25)` spreads retries across a 25% window. We keep it simple here, but if you're running multiple agents against the same API, jitter is worth the one extra line.

The Brain Interface

In Python, we use a base class with a method that raises `NotImplementedError`. Any class that inherits from it must implement the method.

```

73 class Brain:
74     """Base class for LLM providers."""
75
76     def think(self, conversation):
77         """Process conversation, return Thought."""
78         raise NotImplementedError
79
80     def _parse_response(self, content):
81         """Convert API response format to Thought."""
82         text_parts = []
83         tool_calls = []
84         thinking = None
85
86         for block in content:
87             if block["type"] == "thinking":
88                 thinking = block["thinking"]
89             elif block["type"] == "text":
90                 text_parts.append(block["text"])
91             elif block["type"] == "tool_use":
92                 tool_calls.append(ToolCall(
93                     id=block["id"],
94                     name=block["name"],
95                     args=block["input"]
96                 ))
97
98         return Thought(
99             text="\n".join(text_parts) if text_parts else None,
100             tool_calls=tool_calls,
101             thinking=thinking
102         )

```

This is the contract. Every brain must have a `think()` method that takes a conversation and returns a `Thought`. The `_parse_response` method on `Brain` handles the shared response format—both Claude and DeepSeek use Anthropic-compatible APIs, so the parsing logic lives here once.

The FakeBrain (Updated)

Remember `FakeBrain` from Chapter 3? Now it extends `Brain`:

```

class FakeBrain(Brain):
    """Fake brain for testing - returns predictable responses."""

    def __init__(self, responses=None):
        self.responses = responses or [Thought(text="Fake response")]
        self.call_count = 0
        self.last_conversation = None

    def think(self, conversation):
        self.last_conversation = list(conversation)
        if self.call_count < len(self.responses):
            response = self.responses[self.call_count]
            self.call_count += 1
            return response
        return Thought(text="No more responses")

```

This goes in `test_nanocode.py`. It implements the same `think()` interface, so we can swap it for any real brain in tests.

Note that `FakeBrain` extends `Brain` to make the inheritance explicit, but Python uses *duck typing*—any object with a `think(conversation)` method would work. The inheritance is documentation, not a requirement.

The Claude Brain (Refactored)

The Claude class now explicitly extends `Brain`:

```

105 class Claude(Brain):
106     """Claude API implementation."""
107
108     def __init__(self):
109         self.api_key = os.getenv("ANTHROPIC_API_KEY")
110         if not self.api_key:
111             raise ValueError("ANTHROPIC_API_KEY not found in .env")
112         self.model = "claude-sonnet-4-6"
113         self.url = "https://api.anthropic.com/v1/messages"
114
115     def think(self, conversation):
116         headers = {
117             "x-api-key": self.api_key,
118             "anthropic-version": "2023-06-01",
119             "content-type": "application/json"
120         }
121         payload = {
122             "model": self.model,

```

```

123         "max_tokens": 16000,
124         "thinking": {
125             "type": "enabled",
126             "budget_tokens": 10000
127         },
128         "messages": conversation
129     }
130
131     response = request_with_retry(self.url, headers, payload)
132     return self._parse_response(response.json()["content"])

```

Two changes from Chapter 3: `class Claude(Brain)` instead of `class Claude` (signals intent), and we use `request_with_retry` for resilience. The extended thinking payload carries over unchanged. Claude doesn't need its own `_parse_response`—it inherits the one from `Brain`.

The DeepSeek Brain

DeepSeek is a popular alternative—it's cheaper and provides an Anthropic-compatible API.¹ This means we can use the same message format as Claude:

```

135 class DeepSeek(Brain):
136     """DeepSeek API (Anthropic-compatible)."""
137
138     def __init__(self):
139         self.api_key = os.getenv("DEEPSEEK_API_KEY")
140         if not self.api_key:
141             raise ValueError("DEEPSEEK_API_KEY not found in .env")
142         self.model = "deepseek-chat"
143         self.url = "https://api.deepseek.com/anthropic/v1/messages"
144
145     def think(self, conversation):
146         headers = {
147             "x-api-key": self.api_key,
148             "anthropic-version": "2023-06-01",
149             "content-type": "application/json"
150         }
151         payload = {
152             "model": self.model,
153             "max_tokens": 4096,
154             "messages": conversation
155         }

```

¹https://api-docs.deepseek.com/guides/anthropic_api

```
156
157     response = request_with_retry(self.url, headers, payload)
158     return self._parse_response(response.json()["content"])
```

DeepSeek’s Anthropic-compatible endpoint lets us reuse the same headers, message format, and response parsing as Claude. Like Claude, DeepSeek inherits `_parse_response` from `Brain`—each subclass only implements `think()` with its unique payload.



Aside: DeepSeek’s Anthropic-compatible endpoint ignores the anthropic-version header, but including it doesn’t hurt and keeps the code consistent. Since both brains share the same response format, `_parse_response` lives on the base class—each subclass only implements `think()` with its unique payload. If a future provider returns a different format, it can override `_parse_response`.

Both classes return a `Thought`. The Agent doesn’t know or care which brain is thinking.

The BRAINS Registry

We need a way to look up brain classes by name:

```
161 # Available brains
162 BRAINS = {
163     "claude": Claude,
164     "deepseek": DeepSeek,
165 }
```

This dictionary maps string names to classes. We can add new providers (OpenAI, Ollama, etc.) by adding entries here—no other code changes needed.

Here’s how all the pieces fit together:

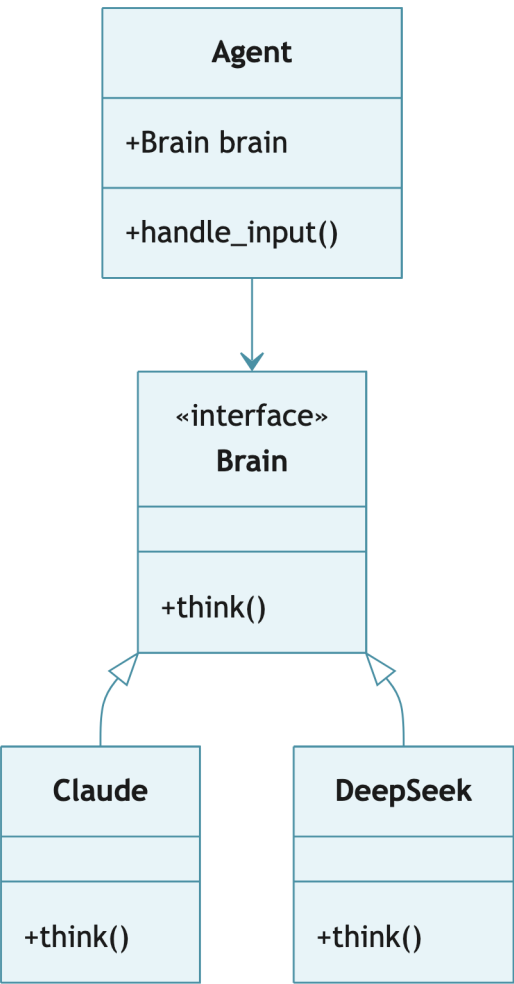


Figure 4. Class diagram showing the Adapter Pattern: Agent depends on the Brain interface, which Claude and DeepSeek implement.

The Agent Class (Updated)

Now the Agent tracks which brain it's using and can switch at runtime:

```

170 class Agent:
171     """A coding agent with switchable brain."""
172
173     def __init__(self, brain, brain_name="claude"):
174         self.brain = brain
175         self.brain_name = brain_name
176         self.conversation = []
177
178     def handle_input(self, user_input):
179         """Handle user input. Returns output string, raises AgentStop to
180         ↪ quit."""
181         if user_input.strip() == "/q":
182             raise AgentStop()
183
184         if user_input.strip() == "/switch":
185             return self._switch_brain()
186
187         if not user_input.strip():
188             return ""
189
190         self.conversation.append({"role": "user", "content": user_input})
191
192         try:
193             thought = self.brain.think(self.conversation)
194             if thought.thinking:
195                 lines = thought.thinking.strip().split("\n")[:5]
196                 for i, line in enumerate(lines):
197                     prefix = " 💬 " if i == 0 else " "
198                     print(f"\033[2m{prefix}{line}\033[0m")
199                 text = thought.text or ""
200                 self.conversation.append({"role": "assistant", "content": text})
201                 return text
202             except Exception as e:
203                 self.conversation.pop()
204                 return f"Error: {e}"

```

New additions:

- **Line 173:** Accept `brain_name` to track which provider we're using.
- **Lines 183-184:** Handle the `/switch` command.
- **Lines 193-197:** Display thinking output, carried forward from Chapter 3's `handle_input`. When the brain returns a `Thought` with thinking content, we print the first five lines dimmed (via ANSI escape codes).

The switch implementation:

```

205     def _switch_brain(self):
206         """Toggle to the next brain."""
207         names = list(BRAINS.keys())
208         idx = names.index(self.brain_name)
209         new_name = names[(idx + 1) % len(names)]
210
211         try:
212             self.brain = BRAINS[new_name]()
213             self.brain_name = new_name
214             return f"Switched to: {new_name}"
215         except ValueError as e:
216             return f"Cannot switch to {new_name}: {e}"

```

This cycles through available brains. If the new brain fails to initialize (missing API key), it returns an error instead of crashing.

Tests for Multi-Brain Support

Let's verify our new functionality:

```

1  def test_agent_stores_brain_name():
2      """Verify agent stores the brain name."""
3      agent = Agent(brain=FakeBrain(), brain_name="claude")
4      assert agent.brain_name == "claude"
5
6
7  def test_brains_registry_has_expected_providers():
8      """Verify BRAINS registry contains expected providers."""
9      assert "claude" in BRAINS
10     assert "deepseek" in BRAINS
11
12
13  def test_switch_command_toggles_brain_name():
14      """Verify /switch updates brain_name."""
15      # Mock BRAINS to use FakeBrain for switching
16      original_brains = BRAINS.copy()
17      BRAINS["claude"] = FakeBrain
18      BRAINS["deepseek"] = FakeBrain
19
20     try:
21         agent = Agent(brain=FakeBrain(), brain_name="claude")
22         result = agent.handle_input("/switch")
23         assert "deepseek" in result
24         assert agent.brain_name == "deepseek"
25     finally:

```

```

26         BRAINS.clear()
27         BRAINS.update(original_brains)

```

Run the tests:

```

1  pytest test_nanocode.py -v

1  test_nanocode.py::test_agent_stores_brain_name PASSED
2  test_nanocode.py::test_brains_registry_has_expected_providers PASSED
3  test_nanocode.py::test_switch_command_toggles_brain_name PASSED

```

The Main Loop (Updated)

The main loop reads the default brain from an environment variable:

```

221 def main():
222     brain_name = os.getenv("NANOCODE_BRAIN", "claude")
223     brain = BRAINS[brain_name]()
224     agent = Agent(brain, brain_name)
225
226     print(f"⚡ Nanocode v0.3")
227     print(f"Commands: /q quit, /switch toggle brain")
228     print(f"Brain: {brain_name}\n")
229
230     while True:
231         try:
232             user_input = input(f"[{agent.brain_name}] > ")
233             output = agent.handle_input(user_input)
234             if output:
235                 print(f"\n{output}\n")
236
237         except (AgentStop, KeyboardInterrupt):
238             print("\nExiting...")
239             break
240
241
242 if __name__ == "__main__":
243     main()

```

The prompt shows the current brain ([claude] >), and it updates after switching.

Setting Up DeepSeek

To use DeepSeek, add your API key to `.env`:

```
1 ANTHROPIC_API_KEY=sk-ant-...
2 DEEPSEEK_API_KEY=sk-...
```

Get a DeepSeek API key at the DeepSeek platform.² It's significantly cheaper than Anthropic for experimentation.

Try It

```
1 python nanocode.py
2 ⚡ Nanocode v0.3
3 Commands: /q quit, /switch toggle brain
4 Brain: claude
5
6 [claude] › Hello!
7
8 Hello! How can I help you today?
9
10 [claude] › /switch
11 Switched to: deepseek
12
13 [deepseek] › Hello!
14
15 Hello! How can I assist you today?
```

Both providers respond. The conversation history carries over—you can start with Claude and continue with DeepSeek.

Or set the default via environment variable:

```
1 NANOCODE_BRAIN=deepseek python nanocode.py
```

²https://platform.deepseek.com/api_keys

“We Just Moved Code Around”

You might think so. But look what “just moving code around” bought us:

- If Anthropic raises prices or DeepSeek gets better, switch with one line.
- Compare model quality by running the same prompt through different providers.
- Use DeepSeek for development, Claude for production.
- FakeBrain extends the same interface, so tests stay trivial.

Dev Tip: In large systems, this pattern is often called an *Anti-Corruption Layer*. It prevents the quirks of an external vendor (Anthropic’s specific JSON format) from “corrupting” your clean internal logic.

Wrapping Up

The agent doesn’t know which LLM it’s talking to, and it shouldn’t. `brain.think()` handles the translation—same idea as that power adapter for international travel.

But right now the agent can only *talk*. Time to give it hands.

Part II: The Hands

Your agent can think, but it can't act. In Part II, you'll give it hands – tools to read, write, and search files – then teach it to remember across sessions and obey safety rails. By Chapter 9, the agent writes code, runs it, reads the errors, and fixes them in a loop.

Chapter 5: The Tool Protocol

Up until now, our agent has been a “Brain in a Jar.” It can think and explain Python, but it can’t touch a single file on your disk.

We’re going to fix that.

How Tools Actually Work

There is a misconception that “Agents” have direct access to your computer. They don’t.

Claude runs on a server in California. It cannot see your hard drive.

To bridge this gap, we use a concept called *Function Calling* (or “Tool Use”). It works like a game of telephone:

You: “Here are the tools you can use: `read_file`, `write_file`.”

Claude: “I would like to run `read_file('main.py')`.”

You (The Python Script): “Okay, I ran it. Here is the content of `main.py`...”

Claude: “Great. Based on that, here is the fix.”

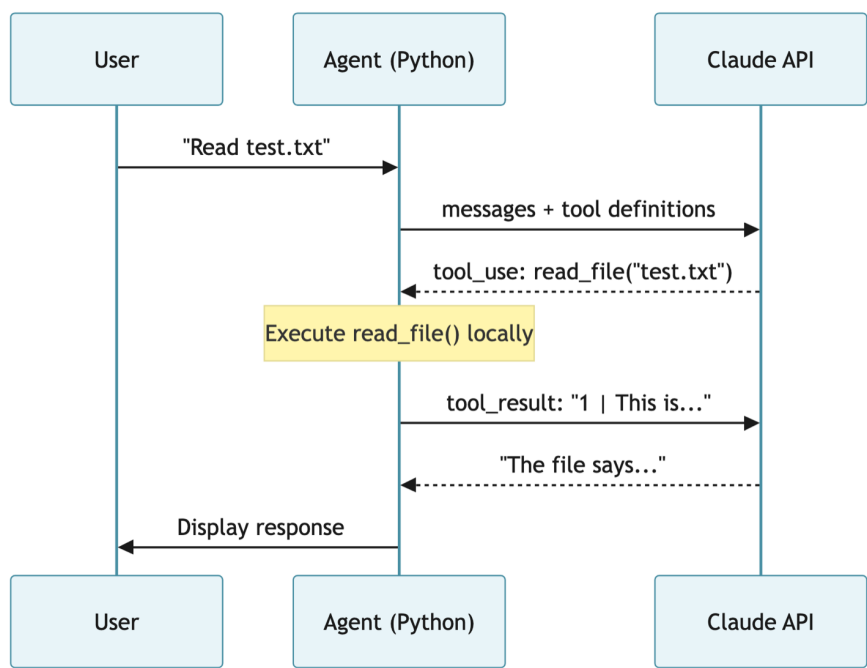


Figure 5. Tool use flow: User asks Agent, Agent sends to Claude with tool definitions, Claude returns tool_use request, Agent executes locally, Agent sends tool_result back, Claude returns final response.

We need to implement three things: tool classes with an `execute()` method, JSON definitions derived from those classes (the “menu” we send to the API), and an *agentic loop* that runs tools until the task is done.



Aside: We keep the Brain base class and DeepSeek adapter from Chapter 4. Since DeepSeek provides an Anthropic-compatible API, both brains support tool execution with the same message format. You can `/switch` between brains and both will execute tools.

Defining the Tool Interface

What does a tool look like? We could use functions, but classes give us a cleaner structure—each tool is self-describing with its name, description, and schema bundled together.

Here’s a test that defines our expectations:

```

1 def test_tool_has_required_attributes():
2     """Verify tool classes have name, description, input_schema."""
3     tool = ReadFile()
4     assert tool.name == "read_file"
5     assert tool.description is not None
6     assert tool.input_schema is not None

```

Each tool is a class with:

- name: String identifier for the API
- description: What the tool does (helps Claude decide when to use it)
- input_schema: JSON Schema describing the parameters
- execute(**args): The actual implementation

The ReadFile Tool

Coding agents have a specific problem: context. If you just dump a file, the LLM doesn't know line numbers. When it tries to edit, it says "Change the line about the loop," which is ambiguous.

Our reader adds line numbers automatically.

The Test:

```

1 def test_read_file_adds_line_numbers():
2     """Verify ReadFile prefixes each line with line numbers."""
3     with tempfile.NamedTemporaryFile(mode='w', suffix='.txt', delete=False) as
4         ↪ f:
5         f.write("line one\nline two\nline three\n")
6         temp_path = f.name
7
8     try:
9         tool = ReadFile()
10        result = tool.execute(temp_path)
11        assert "1 | line one" in result
12        assert "2 | line two" in result
13        assert "3 | line three" in result
14    finally:
15        os.unlink(temp_path)

```

The Code:

```

178 class ReadFile:
179     """Reads a file from the filesystem."""
180     name = "read_file"
181     description = "Reads a file from the filesystem. Use this to examine code."
182     input_schema = {
183         "type": "object",
184         "properties": {
185             "path": {"type": "string", "description": "The path to the file"}
186         },
187         "required": ["path"]
188     }
189
190     def execute(self, path):
191         print(f" → Reading {path}")
192         try:
193             with open(path, 'r', encoding='utf-8') as f:
194                 lines = f.readlines()
195                 numbered_lines = [f"{i+1} | {line}" for i, line in enumerate(lines)]
196                 return "".join(numbered_lines)
197         except Exception as e:
198             return f"Error reading file: {e}"

```

The Walkthrough:

- **Lines 180–188:** Class attributes define the tool's identity. The `input_schema` uses *JSON Schema*—a standard for describing the shape of JSON data. Claude was fine-tuned on this format specifically for tool calling, so it knows how to produce valid arguments that match the schema.
- **Line 190:** The `execute()` method takes the parameters defined in the schema.
- **Line 195:** Enumerate lines and prepend `1 |`, `2 |`, etc. This lets the agent say “Edit line 14.”

Here's how the schema maps to execution: when Claude wants to read a file, it returns a `tool_use` block like this:

```

1 {
2     "type": "tool_use",
3     "id": "toolu_01A",
4     "name": "read_file",
5     "input": {"path": "nanocode.py"}
6 }

```

The input dict maps directly to `execute(**args)`—so `{"path": "nanocode.py"}` becomes `execute(path="nanocode.py")`.

The WriteFile Tool

This is the dangerous one. This tool overwrites files.

The Test:

```

1 def test_write_file_creates_file():
2     """Verify WriteFile creates a file with content."""
3     with tempfile.TemporaryDirectory() as tmpdir:
4         path = os.path.join(tmpdir, "test.txt")
5         tool = WriteFile()
6         result = tool.execute(path, "hello world")
7
8         assert os.path.exists(path)
9         assert "Successfully wrote" in result
10        with open(path) as f:
11            assert f.read() == "hello world"

```

The Code:

```

201 class WriteFile:
202     """Writes content to a file."""
203     name = "write_file"
204     description = "Writes content to a file. OVERWRITES existing content."
205     input_schema = {
206         "type": "object",
207         "properties": {
208             "path": {"type": "string", "description": "The path to the file"},
209             "content": {"type": "string", "description": "The full content to
210                 ↪ write"}
211         },
212         "required": ["path", "content"]

```

```

212     }
213
214     def execute(self, path, content):
215         print(f" → Writing {path}")
216         try:
217             with open(path, 'w', encoding='utf-8') as f:
218                 f.write(content)
219             return f"Successfully wrote {len(content)} characters to {path}"
220         except Exception as e:
221             return f"Error writing file: {e}"

```

The return message confirms success—Claude needs feedback to know the operation worked.

Tool Helpers

We need two helper functions: one to find a tool by name, and one to generate API definitions.

```

226 def get_tool(tools, name):
227     """Find a tool by name, or None if not found."""
228     return next((t for t in tools if t.name == name), None)
229
230
231 def tool_definitions(tools):
232     """Return tool definitions for the API."""
233     return [
234         {"name": t.name, "description": t.description, "input_schema":
235          ↪ t.input_schema}
236         for t in tools
237     ]

```

The `tool_definitions()` function extracts just the metadata Claude needs—no `execute` method included. This is the “menu” we send to the API.

Now instantiate the tools:

```

241 tools = [ReadFile(), WriteFile()]

```

With tools defined and helpers in place, here’s the full picture of our agent’s architecture:

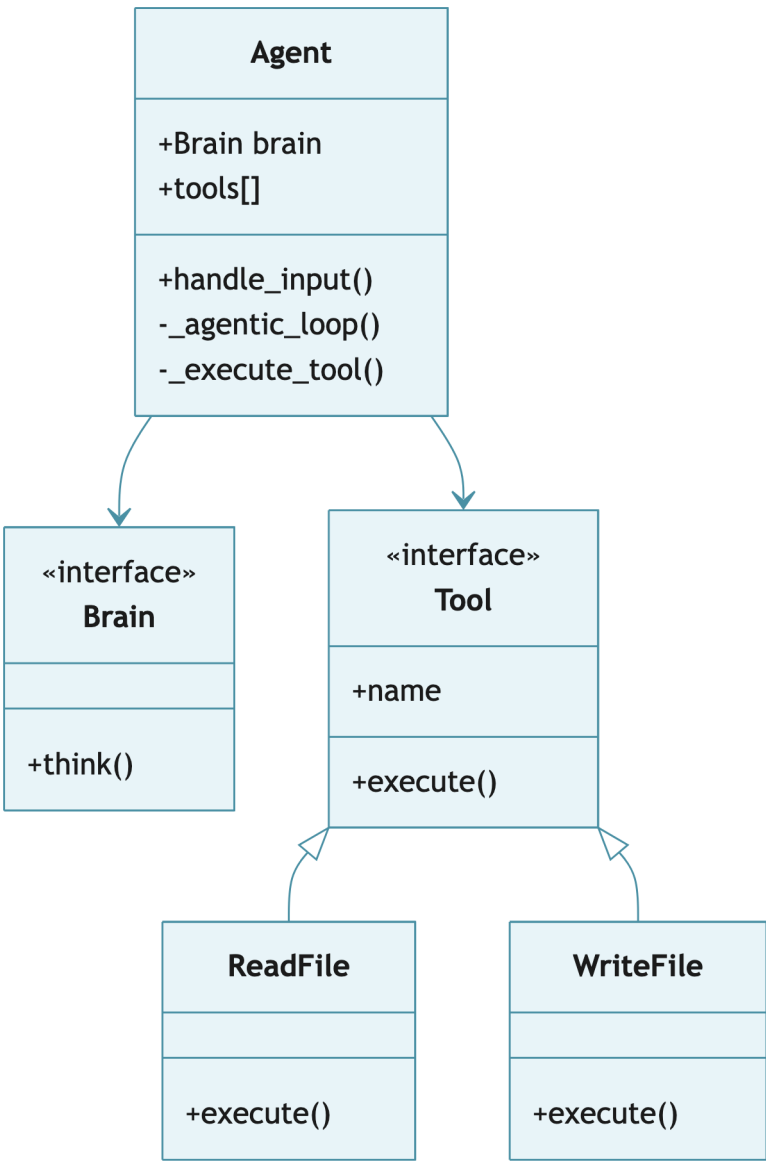


Figure 6. Class diagram showing the Agent’s relationships: Agent depends on Brain (interface) and the Tool convention. ReadFile and WriteFile follow the Tool convention (plain classes, no base class).

Updating the Thought Class

Before we wire up the agentic loop, we need to update our Thought class from Chapter 3. Claude’s tool protocol requires us to send back the *exact* response

format in the conversation history. We add `raw_content` to preserve it:

```

62 class Thought:
63     """Standardized response from any Brain."""
64
65     def __init__(self, text=None, tool_calls=None, raw_content=None,
66         ↪ thinking=None):
67         self.text = text # str or None
68         self.tool_calls = tool_calls or [] # list of ToolCall
69         self.raw_content = raw_content # original API response for message
70         ↪ history
71         self.thinking = thinking # str or None

```

The `raw_content` field stores Claude’s exact response. When we append the assistant message to conversation history, we use `raw_content` instead of reconstructing it from `text`.

Why `raw_content`? Claude’s tool protocol has a strict requirement: when you append an assistant message to the conversation history, you must include the *exact* content array that Claude returned—including both text blocks and `tool_use` blocks. If you reconstruct it (e.g., just storing the text), Claude loses track of which tool calls are pending and returns errors like “`tool_use_id` not found.” The `raw_content` field preserves this exact format.

Updating the Claude Class

The Claude class needs to accept tool definitions and include them in API requests:

```

107 class Claude(Brain):
108     """Claude API - the brain of our agent."""
109
110     def __init__(self, tools=None):
111         self.tools = tools or []
112         self.api_key = os.getenv("ANTHROPIC_API_KEY")
113         if not self.api_key:
114             raise ValueError("ANTHROPIC_API_KEY not found in .env")
115         self.model = "claude-sonnet-4-6"
116         self.url = "https://api.anthropic.com/v1/messages"
117
118     def think(self, conversation):
119         headers = {

```

```

120         "x-api-key": self.api_key,
121         "anthropic-version": "2023-06-01",
122         "content-type": "application/json"
123     }
124     payload = {
125         "model": self.model,
126         "max_tokens": 16000,
127         "thinking": {
128             "type": "enabled",
129             "budget_tokens": 10000
130         },
131         "messages": conversation
132     }
133     if self.tools:
134         payload["tools"] = self.tools
135
136     response = request_with_retry(self.url, headers, payload)
137     return self._parse_response(response.json()["content"])

```

Line 110: Accept tools in the constructor. **Lines 133-134:** Only include tools in the payload if we have any.

Apply the same change to DeepSeek: add `tools=None` to `__init__`, store as `self.tools`, and add `if self.tools: payload["tools"] = self.tools` in `think()`. The BRAINS registry passes `tools=` to whichever brain it instantiates, so both constructors need the same signature.

The Brain base class's `_parse_response` method now preserves `raw_content` to keep the exact response format for conversation history:

```

81     def _parse_response(self, content):
82         """Convert API response format to Thought."""
83         text_parts = []
84         tool_calls = []
85         thinking = None
86
87         for block in content:
88             if block["type"] == "thinking":
89                 thinking = block["thinking"]
90             elif block["type"] == "text":
91                 text_parts.append(block["text"])
92             elif block["type"] == "tool_use":
93                 tool_calls.append(ToolCall(
94                     id=block["id"],
95                     name=block["name"],
96                     args=block["input"]
97                 ))

```

```

98
99         return Thought(
100             text="\n".join(text_parts) if text_parts else None,
101             tool_calls=tool_calls,
102             raw_content=content,
103             thinking=thinking
104         )

```

Line 102: Store the original content array as `raw_content`. Claude expects this exact format when we send the assistant message back. The `raw_content` includes thinking blocks, which Claude uses for continuity across turns.

The Agent Class with Tools

The Agent class needs an *agentic loop*—a while True that keeps calling the brain and executing tools until the brain stops asking for more.

The Test:

```

1  def test_agentic_loop_executes_tool_calls():
2      """Verify agentic loop executes tool calls and continues."""
3      # Brain returns a tool call, then a final response
4      brain = FakeBrain(responses=[
5          Thought(
6              text="Let me read that file.",
7              tool_calls=[ToolCall(id="1", name="read_file", args={"path":
8                  ↪ temp_path})],
9              raw_content=[
10                 {"type": "text", "text": "Let me read that file."},
11                 {"type": "tool_use", "id": "1", "name": "read_file", "input":
12                     ↪ {"path": temp_path}}
13             ]
14         ),
15         Thought(
16             text="The file contains test content.",
17             raw_content=[{"type": "text", "text": "The file contains test
18                 ↪ content."}]
19         )
20     ])
21     agent = Agent(brain=brain, tools=tools)
22     result = agent.handle_input("Read the file")
23
24     assert brain.call_count == 2 # Called twice (tool call + final)

```

The pattern: the agent calls the brain, executes any tool calls, sends results back, and repeats until the brain stops requesting tools.

The Code:

```

246 class Agent:
247     """A coding agent with tools."""
248
249     def __init__(self, brain, tools, brain_name="claude"):
250         self.brain = brain
251         self.tools = list(tools) # Copy the tools list
252         self.brain_name = brain_name
253         self.conversation = []
254
255     def handle_input(self, user_input):
256         """Handle user input. Returns output string, raises AgentStop to
257         ↪ quit."""
258         if user_input.strip() == "/q":
259             raise AgentStop()
260
261         if user_input.strip() == "/switch":
262             return self._switch_brain()
263
264         if not user_input.strip():
265             return ""
266
267         self.conversation.append({"role": "user", "content": user_input})
268
269         try:
270             return self._agentic_loop()
271         except Exception as e:
272             self.conversation.pop() # Remove failed user message
273             return f"Error: {e}"

```

The `handle_input` method is similar to Chapter 3, but now delegates to `_agentic_loop()`.

The Agentic Loop:

```

287 def _agentic_loop(self):
288     """Process brain responses, executing tools until done."""
289     output_parts = []
290
291     while True:
292         thought = self.brain.think(self.conversation)
293
294         # Display thinking
295         if thought.thinking:
296             lines = thought.thinking.strip().split("\n")[:5]
297             for i, line in enumerate(lines):
298                 prefix = "  🗨️ " if i == 0 else "      "
299                 print(f"\033[2m{prefix}{line}\033[0m")
300
301             # Store raw content for message history (Claude expects this format)
302             self.conversation.append({"role": "assistant", "content":
303                                     ↪ thought.raw_content})
304
305             # Collect text output
306             if thought.text:
307                 output_parts.append(thought.text)
308
309             # Check for tool calls
310             if not thought.tool_calls:
311                 break # No more tools to execute
312
313             # Execute tools and collect results
314             tool_results = []
315             for tool_call in thought.tool_calls:
316                 result = self._execute_tool(tool_call.name, tool_call.args)
317                 tool_results.append({
318                     "type": "tool_result",
319                     "tool_use_id": tool_call.id,
320                     "content": result
321                 })
322
323             self.conversation.append({"role": "user", "content": tool_results})
324
325     return "\n".join(output_parts)

```

The Walkthrough:

- **Line 291:** Loop forever until no more tool calls.
- **Lines 294-299:** Display thinking output, moved here from `handle_input` so it runs on every iteration of the agentic loop.
- **Line 302:** Append `raw_content` to conversation—this is critical. Claude expects to see its exact response format. Notice this replaces the plain

text string from Chapter 3 (`{"content": text}`) with the raw API content list (`{"content": thought.raw_content}`).



Warning: If you're carrying forward Chapter 3 tests that assert `conversation[1]["content"]` is a string, update them—content is now a list of dicts. For example, `assert agent.conversation[1]["content"] == "Hello"` becomes `assert agent.conversation[1]["content"][0]["text"] == "Hello"` (or just check `thought.text` instead).

- **Lines 308-309:** If no tool calls, we're done. Break out of the loop.
- **Lines 312-319:** Execute each tool, format results with `tool_use_id` matching the request.
- **Line 322:** Send results back as a “user” message. Claude's protocol alternates strictly between user and assistant roles. Tool results come from your code—the user's side—so they go in a “user” message.



Warning: Notice that `_agentic_loop` has no iteration limit. If the model hallucinates tool calls in a loop, it will keep running forever—burning your API budget. We'll add a `max_iterations` guard in Chapter 9 when feedback loops make this risk real. Until then, keep an eye on it—a runaway loop is real money.

Tool Execution:

```

326 def _execute_tool(self, name, args):
327     """Execute a tool by name with given arguments."""
328     tool = get_tool(self.tools, name)
329     if tool is None:
330         return f"Error: Tool '{name}' not found"
331     try:
332         return tool.execute(**args)
333     except TypeError as e:
334         return f"Error: Invalid arguments - {e}"

```

The `**args` unpacks the dictionary into keyword arguments, matching the tool's `execute()` signature.

The Main Loop

The main loop is straightforward—create the brain with tool definitions, create the agent with tools:

```

339 def main():
340     brain_name = os.getenv("NANOCODE_BRAIN", "claude")
341     brain = BRAINS[brain_name](tools=tool_definitions(tools))
342     agent = Agent(brain=brain, tools=tools, brain_name=brain_name)
343
344     print(f"⚡ Nanocode v0.4 (Tools Enabled)")
345     print(f"Commands: /q quit, /switch toggle brain")
346     print(f"Brain: {brain_name}\n")
347
348     while True:
349         try:
350             user_input = input(f"[{agent.brain_name}] > ")
351             output = agent.handle_input(user_input)
352             if output:
353                 print(f"\n{output}\n")
354
355             except (AgentStop, KeyboardInterrupt):
356                 print("\nExiting...")
357                 break
358
359 if __name__ == "__main__":
360     main()
361
```

Notice line 341: `BRAINS[brain_name]` looks up the brain class, and `tool_definitions(tools)` extracts the JSON schemas.

Since `Claude.__init__` now accepts `tools=`, we also need to update `_switch_brain` from Chapter 4 to pass tool definitions:

```
def _switch_brain(self):
    """Toggle to the next brain."""
    names = list(BRAINS.keys())
    idx = names.index(self.brain_name)
    new_name = names[(idx + 1) % len(names)]

    try:
        self.brain = BRAINS[new_name](tools=tool_definitions(self.tools))
        self.brain_name = new_name
        return f"Switched to: {new_name}"
    except ValueError as e:
        return f"Cannot switch to {new_name}: {e}"
```

Without this, switching brains would create a brain with no tools.

Test It

Create a dummy file called `test.txt`:

```
1 This is a secret message.
```

Run `nanocode.py`:

```
1 python nanocode.py
2
3 › Read test.txt
4
5 Let me read that file.
6
7 The file says: "This is a secret message."
```

That's a round trip: your script sent the request to Claude, Claude asked for a tool, your script executed it locally, and Claude summarized the result.

Dev Tip: Right now, our agent runs whatever tool Claude asks for. If Claude asks to `write_file('/etc/passwd')`, it will try. We'll build a "Plan Mode"

safety harness to prevent this.



Aside: This `while True` loop—call the brain, execute tools, feed results back—is the same architecture that powers production agents like Claude Code. The sophistication in production isn’t a different loop—it’s what happens when the loop *fails*. The loop itself is what you just wrote.

Wrapping Up

The agent can touch the world now. It reads files, writes files, and the `while True` in `_agentic_loop` keeps calling the brain until the job is done.

There’s an obvious problem, though: the agent forgets everything when you restart it. We fix that next with persistent memory—a Markdown file, not a vector database.

Chapter 6: The Scratchpad (Memory)

Our agent currently has a fatal flaw.

If you have a great conversation where you explain exactly how you like your variables named, and then you hit Ctrl+C, that knowledge is gone forever. The next time you run `nanocode.py`, the agent is a blank slate.

In AI engineering, this is the “Persistence Problem.”

Most frameworks solve this by selling you a *vector database* (a complex way to store text as math). We are not going to use that. Vector databases are overkill for a single developer.

Instead, we are going to use the most reliable database in the world: **A Markdown file.**

The “Zero Magic” Memory

Up to now, “memory” meant the conversation list—messages that accumulate during a session and vanish when the process exits. What we’re building here is different: *persistent* memory that survives between sessions.

We will give our agent a “Scratchpad”—a file called `.nanocode/memory.md`.

Before every turn, we read this file and inject it into the agent’s brain via the *system prompt*. During a turn, the agent can use a tool to rewrite it. The file stays on disk between sessions.

This creates a “Self-Modifying System Instruction”—the agent can update its own operating instructions, and those instructions persist across sessions.

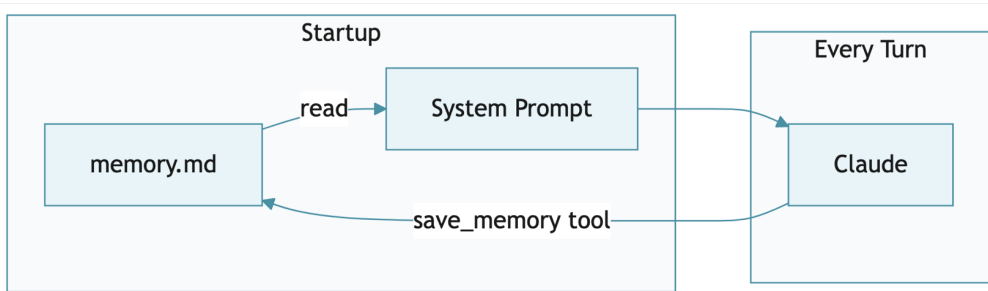


Figure 7. Memory flow: On startup, memory.md is read into the system prompt. Every turn, Claude can use save_memory to update the file, which persists for future sessions.

The Memory Class

The Memory class keeps the logic clean and testable.

The Test:

```

1  def test_memory_creates_default_file():
2      """Verify Memory creates file with default content if missing."""
3      with tempfile.TemporaryDirectory() as tmpdir:
4          path = os.path.join(tmpdir, "memory.md")
5          memory = Memory(path=path)
6
7          assert os.path.exists(path)
8          assert "Nanocode" in memory.content
9
10
11 def test_memory_save_updates_content_and_file():
12     """Verify Memory.save() updates both content and file."""
13     with tempfile.TemporaryDirectory() as tmpdir:
14         path = os.path.join(tmpdir, "memory.md")
15         memory = Memory(path=path)
16
17         memory.save("New content")
18
19         assert memory.content == "New content"
20         with open(path) as f:
21             assert f.read() == "New content"
  
```

The Code:

```

74 class Memory:
75     """Persistent scratchpad for the agent."""
76
77     def __init__(self, path=".nanocode/memory.md"):
78         self.path = path
79         self._ensure_exists()
80         self.content = self._load()
81
82     def _ensure_exists(self):
83         """Create memory file with default content if needed."""
84         os.makedirs(os.path.dirname(self.path), exist_ok=True)
85         if not os.path.exists(self.path):
86             default = "I am Nanocode, a helpful coding assistant.\n"
87             with open(self.path, "w") as f:
88                 f.write(default)
89
90     def _load(self):
91         """Load content from disk."""
92         with open(self.path, 'r') as f:
93             return f.read()
94
95     def save(self, content):
96         """Update memory content and persist to disk."""
97         self.content = content
98         with open(self.path, 'w') as f:
99             f.write(content)

```

The Walkthrough:

- **Lines 78-80:** On init, ensure the file exists and load its content.
- **Lines 82-88:** Create the `.nanocode` directory and a default memory file if needed.
- **Lines 95-99:** `save()` updates both the in-memory content and the file on disk.

The ToolContext Class

Now we have a problem. The `SaveMemory` tool needs access to the `Memory` object—but tools in Chapter 5 only received their explicit arguments. We need a way to pass shared state to tools.

We solve this with `ToolContext`—a simple container that holds whatever state tools might need.

```

104 class ToolContext:
105     """What tools need to know about the agent's state."""
106
107     def __init__(self, memory=None):
108         self.memory = memory # Memory object or None

```

For now, it just holds memory. We'll add more shared state as the agent grows.

This also means our existing tools need to accept a context parameter. Update `ReadFile` and `WriteFile`:

```

def execute(self, context, path): # ReadFile
def execute(self, context, path, content): # WriteFile

```

They don't use context yet, but `SaveMemory` will—and future tools might. Changing the interface now saves us from touching every tool call site twice.



Warning: This changes the calling convention for all tools. If you wrote tests for Chapter 5, update them to pass a `ToolContext()` as the first argument. For example, `ReadFile().execute(path=temp_path)` becomes `ReadFile().execute(ToolContext(), path=temp_path)`.

The SaveMemory Tool

We need a tool that lets the agent update its memory. Unlike `WriteFile` (which edits code), this tool edits the agent's *mind*.

The Test:

```

1  def test_save_memory_updates_memory():
2      """Verify SaveMemory updates the Memory object."""
3      with tempfile.TemporaryDirectory() as tmpdir:
4          memory = Memory(path=os.path.join(tmpdir, "memory.md"))
5          tool = SaveMemory()
6          context = ToolContext(memory=memory)
7
8          result = tool.execute(context, "Updated preferences")
9
10         assert "successfully" in result.lower()
11         assert memory.content == "Updated preferences"

```

The Code:

```

269 class SaveMemory:
270     """Updates the agent's internal memory/scratchpad."""
271     name = "save_memory"
272     description = "Updates your internal memory/scratchpad. Use this to
273     ↪ remember user preferences."
274     input_schema = {
275         "type": "object",
276         "properties": {
277             "content": {"type": "string", "description": "The full text to
278             ↪ save."}
279         },
280         "required": ["content"]
281     }
282     def execute(self, context, content):
283         print(f" → Saving memory")
284         if context.memory is None:
285             return "Error: Memory not available"
286         context.memory.save(content)
287         return "Memory updated successfully."

```

The tool accesses the Memory object through `context.memory`. If memory isn't available (maybe we're testing without it), it returns an error instead of crashing.

Add it to the tools list:

```

306 tools = [ReadFile(), WriteFile(), SaveMemory()]

```

Updating the Claude Class

The Claude class needs to inject memory content as a *system prompt*—instructions that shape the model’s behavior before any user messages.

```

146 class Claude(Brain):
147     """Claude API - the brain of our agent."""
148
149     def __init__(self, memory=None, tools=None):
150         self.memory = memory
151         self.tools = tools or []
152         self.api_key = os.getenv("ANTHROPIC_API_KEY")
153         if not self.api_key:
154             raise ValueError("ANTHROPIC_API_KEY not found in .env")
155         self.model = "claude-sonnet-4-6"
156         self.url = "https://api.anthropic.com/v1/messages"
157
158     def think(self, conversation):
159         headers = {
160             "x-api-key": self.api_key,
161             "anthropic-version": "2023-06-01",
162             "content-type": "application/json"
163         }
164         payload = {
165             "model": self.model,
166             "max_tokens": 16000,
167             "thinking": {
168                 "type": "enabled",
169                 "budget_tokens": 10000
170             },
171             "messages": conversation
172         }
173         if self.memory:
174             payload["system"] = self.memory.content
175         if self.tools:
176             payload["tools"] = self.tools
177
178         response = request_with_retry(self.url, headers, payload)
179         return self._parse_response(response.json()["content"])

```

Key changes:

- **Line 149:** Accept memory in the constructor.
- **Lines 173-174:** If memory exists, inject its content as the system parameter.

Every time Claude thinks, it “hears” the memory file in its head via the system prompt.

Apply the same change to DeepSeek: add `memory=None` to `__init__`, store as `self.memory`, and add `if self.memory: payload["system"] = self.memory.content` in `think()`. The BRAINS registry passes `memory=` when constructing any brain, so both constructors need the same signature.

Crafting the System Prompt

Our default memory content is minimal: "I am Nanocode, a coding agent...". In practice, you'd pack it with role instructions, behavioral guidelines, and tool usage hints:

```
I am Nanocode, a coding agent.
```

Rules

- Always read a file before editing it.
- Run tests after every code change.
- Never modify files outside the project directory.

Style

- Use clear variable names. No single-letter variables.
- Prefer small, focused functions.

Dev Tip: Because memory is the system prompt, and the agent can write to memory, it can evolve its own rules over time. Worth monitoring—check `.nanocode/memory.md` occasionally to see what your agent has decided to remember.



Aside: Some production agents take this further—running a background pass after each query to automatically extract memories into structured files. The agent builds its own knowledge base without the user saying “remember this.” Our scratchpad is the same mechanism, just manual.



Warning: This also means a malicious or confused prompt can persist bad instructions across sessions via `save_memory`. If the agent starts behaving strangely, check `.nanocode/memory.md` for anything unexpected and delete the file to reset.

Updating the Agent Class

The Agent now passes memory to tools via `ToolContext`:

```

311 class Agent:
312     """A coding agent with tools and memory."""
313
314     def __init__(self, brain, tools, memory=None, brain_name="claude"):
315         self.brain = brain
316         self.tools = list(tools) # Copy the tools list
317         self.memory = memory
318         self.brain_name = brain_name
319         self.conversation = []

```

And in `_execute_tool`:

```

379 def _execute_tool(self, name, args):
380     """Execute a tool by name with given arguments."""
381     tool = get_tool(self.tools, name)
382     if tool is None:
383         return f"Error: Tool '{name}' not found"
384     try:
385         context = ToolContext(memory=self.memory)
386         return tool.execute(context, **args)
387     except TypeError as e:
388         return f"Error: Invalid arguments - {e}"

```

Line 385: Create a `ToolContext` with the memory object. The `SaveMemory` tool can now access it.

The Main Loop (Updated)

Wire it all together:

```

406 def main():
407     brain_name = os.getenv("NANOCODE_BRAIN", "claude")
408     memory = Memory()
409     brain = BRAINS[brain_name](memory=memory, tools=tool_definitions(tools))
410     agent = Agent(brain=brain, tools=tools, memory=memory,
411                  ↪ brain_name=brain_name)
412
413     print(f"⚡ Nanocode v0.5 (Memory Enabled)")
414     print(f"Commands: /q quit, /switch toggle brain")
415     print(f"Brain: {brain_name}")
416
417     while True:
418         try:
419             user_input = input(f"[{agent.brain_name}] > ")
420             output = agent.handle_input(user_input)
421             if output:
422                 print(f"\n{output}\n")
423
424             except (AgentStop, KeyboardInterrupt):
425                 print("\nExiting...")
426                 break
427
428 if __name__ == "__main__":
429     main()

```

Lines 408-410: Create Memory and use BRAINS registry to instantiate the brain, passing memory to both Claude (for the system prompt) and Agent (for tool access).

Also update `_switch_brain` to pass `memory=self.memory` when constructing the new brain—otherwise switching brains would lose the system prompt:

```

def _switch_brain(self):
    # ... cycle to next brain ...
    self.brain = BRAINS[self.brain_name](
        memory=self.memory, tools=tool_definitions(self.tools)
    )

```

Testing Persistence

Two sessions, one memory file.

Session 1:

```
1 python nanocode.py
2
3 › My name is Alice. Please remember that I prefer TypeScript over Python for
  → frontend.
4
5 [Tool Use: save_memory]
6
7 Understood, Alice. I have updated my memory.
```

Session 2 (Restart the script):

```
1 python nanocode.py
2
3 › Who am I and what language do I like?
4
5 You are Alice, and you prefer TypeScript for frontend development.
```

As you work on a project, the agent builds up a context file of decisions, weird bugs, and style preferences.

And because it's just a file, you can open `.nanocode/memory.md` in VS Code and edit it yourself. Paste in your team's coding standards and they'll be in the system prompt on the next turn.



Warning: The contents of `memory.md` are sent with every single API request. Never paste sensitive information (API keys, passwords, customer data) into the scratchpad—it will be transmitted to the LLM provider. Also keep it concise: a 50-page document will drain your API budget quickly.

The `.nanocode/` directory is created automatically on startup when `Memory()` is instantiated—before the agent processes any input. To reset the agent's memory, delete the file: `rm .nanocode/memory.md`. It will be recreated with the default content on the next startup. We added `.nanocode/` to `.gitignore` in Chapter 1 to prevent accidentally committing memory contents to version control.

Wrapping Up

Three plain classes—`Memory`, `ToolContext`, `SaveMemory`—and the agent can rewrite its own brain between sessions. A Markdown file you can read and

edit yourself.

One thing we haven't addressed: the agent can still overwrite any file it wants. It needs a safety switch.

Chapter 7: The Safety Harness (Plan Mode)

We have built a powerful machine. It can read code, write code, and overwrite your files.

Imagine we ask our agent to “Refactor all files.” It might misunderstand and delete your entire project. Or, more subtly, it might start “fixing” code that wasn’t broken, introducing bugs while you are getting coffee.

In robotics, every heavy machine has a “Big Red Button.” In software, we need a “Safety Harness.”

The fix is “Plan Mode”—a safety harness that controls which tools the agent can see.

The Concept

The Agent gets a mode attribute: either “plan” or “act”. In plan mode, we remove write tools from the LLM’s menu entirely. It can’t call `write_file` because it doesn’t know `write_file` exists. Instead, it gets a dedicated `write_plan` tool that only writes to `PLAN.md`. In act mode, the full menu is restored.

This creates a safe workflow: you ask the agent to plan a task, it reads the code and writes `PLAN.md`. You review the plan. If you like it, switch to act mode and say “Follow the plan.”

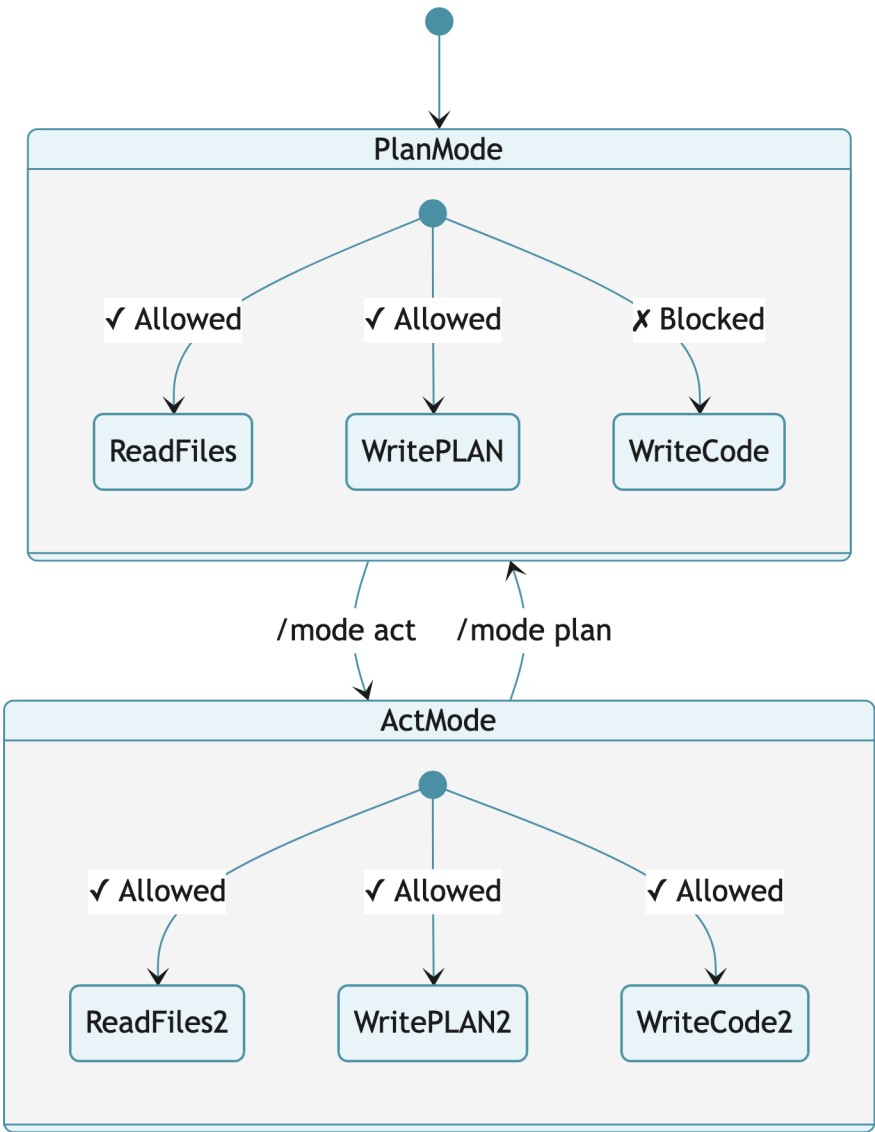


Figure 8. Plan Mode state diagram: In Plan Mode, reading files and writing PLAN.md are allowed, but writing code is blocked. In Act Mode, all operations are allowed. Users toggle between modes with /mode command.

Tests First

We define the behavior before writing the implementation:

```

1  def test_agent_defaults_to_plan_mode():
2      """Verify agent starts in plan mode by default."""
3      agent = Agent(brain=FakeBrain(), tools=tools)
4      assert agent.mode == "plan"
5
6
7  def test_plan_mode_hides_write_file():
8      """Verify plan mode does not expose write_file to the brain."""
9      agent = Agent(brain=FakeBrain(), tools=tools, mode="plan")
10     tool_names = [t["name"] for t in agent.brain.tools]
11
12     assert "write_file" not in tool_names
13     assert "write_plan" in tool_names
14
15
16  def test_act_mode_shows_all_tools():
17      """Verify act mode exposes all tools to the brain."""
18      agent = Agent(brain=FakeBrain(), tools=tools, mode="act")
19      tool_names = [t["name"] for t in agent.brain.tools]
20
21     assert "write_file" in tool_names
22
23
24  def test_mode_command_switches_to_act():
25      """Verify /mode act switches to act mode."""
26      agent = Agent(brain=FakeBrain(), tools=tools, mode="plan")
27      result = agent.handle_input("/mode act")
28
29     assert agent.mode == "act"
30     assert "ACT" in result

```

Notice what we're testing: not that a tool call gets blocked, but that the tool never appears on the menu. The LLM can't call what it can't see.

The WritePlan Tool

Instead of letting WriteFile handle PLAN.md as a special case, we create a dedicated tool for planning:

```

274 class WritePlan:
275     """Saves a plan to PLAN.md."""
276     name = "write_plan"
277     plan_safe = True
278     description = "Saves a plan to PLAN.md. Use this to outline your approach
↳ before making changes."
279     input_schema = {
280         "type": "object",
281         "properties": {
282             "content": {"type": "string", "description": "The plan content in
↳ markdown"}
283         },
284         "required": ["content"]
285     }
286
287     def execute(self, context, content):
288         print(" → Writing PLAN.md")
289         try:
290             with open("PLAN.md", 'w', encoding='utf-8') as f:
291                 f.write(content)
292             return "Plan saved to PLAN.md"
293         except Exception as e:
294             return f"Error saving plan: {e}"

```

No path argument, no mode checks. It writes PLAN.md and nothing else. And plan_safe = True means the brain can see it in plan mode.

Meanwhile, WriteFile marks itself as plan_safe = False:

```

250 class WriteFile:
251     """Writes content to a file."""
252     name = "write_file"
253     plan_safe = False
254     description = "Writes content to a file."
255     input_schema = {
256         "type": "object",
257         "properties": {
258             "path": {"type": "string", "description": "The path to the file"},
259             "content": {"type": "string", "description": "The full content to
↳ write"}
260         },
261         "required": ["path", "content"]
262     }
263
264     def execute(self, context, path, content):
265         print(f" → Writing {path}")
266         try:
267             with open(path, 'w', encoding='utf-8') as f:

```

```
268         f.write(content)
269     return f"Successfully wrote {len(content)} characters to {path}"
270 except Exception as e:
271     return f"Error writing file: {e}"
```

No guardrail logic inside the tool. The guardrail is the tag.

One List, Two Views

Every tool carries a `plan_safe` attribute. Add `plan_safe = True` to `ReadFile` and `SaveMemory` (read-only operations safe for plan mode) and `plan_safe = False` to `WriteFile` (mutates the filesystem). The agent filters the list based on the current mode:

```
335 tools = [ReadFile(), WritePlan(), SaveMemory(), WriteFile()]
```

One flat list. The agent shows the brain only `plan_safe = True` tools in plan mode, and all tools in act mode.



Aside: `SaveMemory` is `plan_safe = True`. Memory updates are about the agent's internal state, not your codebase. Letting the agent remember user preferences ("I prefer TypeScript") even while planning is useful.

Telling the Brain What Mode It's In

Hiding `write_file` from the menu prevents the LLM from writing code. But it doesn't tell the LLM what to do *instead*. Without guidance, the LLM sees no file-writing tool and just dumps its plan as conversation text – technically safe, but useless for our plan-then-act workflow.

We need the system prompt to close the loop. Until now, the Brain read `memory.content` directly as the system prompt. But the Agent knows things the Brain doesn't – like which mode we're in. So we'll have the Agent control the system prompt, just like it controls the tool menu.

Here's the change in Claude – apply the same to DeepSeek:

```

150     def __init__(self, memory=None, tools=None):
151         self.memory = memory
152         self.tools = tools or []
153         self.system = None
154         self.api_key = os.getenv("ANTHROPIC_API_KEY")

175     if self.system:
176         payload["system"] = self.system

```

The Brain no longer decides what the system prompt says – it just sends whatever `self.system` contains. The Agent fills it in.

The Agent Class (Updated)

The Agent now has a `mode` attribute, a `_tools_for_mode()` method that swaps the brain's tool menu, and a `_build_system_prompt()` method that injects mode context into the system prompt:

```

340 class Agent:
341     """A coding agent with tools, memory, and safety mode."""
342
343     def __init__(self, brain, tools, memory=None, mode="plan",
344         ↪ brain_name="claude"):
345         self.brain = brain
346         self.tools = list(tools) # Copy the tools list
347         self.memory = memory
348         self.mode = mode # "plan" or "act"
349         self.brain_name = brain_name
350         self.conversation = []
351         self.brain.tools = self._tools_for_mode()
352         self.brain.system = self._build_system_prompt()
353
354     def _build_system_prompt(self):
355         """Build system prompt from memory and current mode."""
356         parts = [self.memory.content] if self.memory else []
357         if self.mode == "plan":
358             parts.append(
359                 "You are in PLAN mode. You cannot write code files. "
360                 "Use write_plan to save your plans to PLAN.md."
361             )
362         return "\n".join(parts)

```

```

362
363     def _tools_for_mode(self):
364         """Return tool definitions based on current mode."""
365         if self.mode == "act":
366             return tool_definitions(self.tools)
367         return tool_definitions([t for t in self.tools if t.plan_safe])

```

Two lines in the constructor do the work: line 350 sets the tool menu, line 351 sets the system prompt. Both are rebuilt from the current mode.

`_build_system_prompt()` starts with the memory content (the agent's persistent identity and rules) and appends mode context when in plan mode. In act mode, the LLM just gets the memory – no restrictions. Every mode switch rebuilds both the tool menu and the system prompt—the LLM's constraints stay in sync.



Warning: Add `import sys` at the top of `nanocode.py`—the `sys.argv` reference in the main loop needs it.

Wire the `/mode` command into `handle_input`, right after the `/switch` check:

```

def handle_input(self, user_input):
    # ... /q and /switch handlers ...
    if user_input.strip().startswith("/mode"):
        return self._handle_mode_command(user_input)
    # ... rest of method ...

```

The mode switching implementation updates both the brain's menu and system prompt on every switch:

```

392 def _handle_mode_command(self, user_input):
393     """Handle /mode command to switch between plan and act."""
394     parts = user_input.strip().split()
395     if len(parts) > 1 and parts[1] == "act":
396         self.mode = "act"
397         self.brain.tools = self._tools_for_mode()
398         self.brain.system = self._build_system_prompt()
399         return "⚠ Switched to ACT MODE (Writing Enabled)"
400     else:
401         self.mode = "plan"
402         self.brain.tools = self._tools_for_mode()
403         self.brain.system = self._build_system_prompt()
404         return "🛡 Switched to PLAN MODE (Code Read-Only)"

```

And `_execute_tool` stays simple—no mode logic needed:

```

445 def _execute_tool(self, name, args):
446     """Execute a tool by name with given arguments."""
447     tool = get_tool(self.tools, name)
448     if tool is None:
449         return f"Error: Tool '{name}' not found"
450     try:
451         context = ToolContext(memory=self.memory)
452         return tool.execute(context, **args)
453     except TypeError as e:
454         return f"Error: Invalid arguments - {e}"

```

The Main Loop (Updated)

The main loop parses the `--act` flag and shows the mode in the prompt:

```

473 def main():
474     # Parse mode from CLI
475     mode = "act" if len(sys.argv) > 1 and sys.argv[1] == "--act" else "plan"
476     brain_name = os.getenv("NANOCODE_BRAIN", "claude")
477
478     memory = Memory()
479     brain = BRAINS[brain_name](memory=memory, tools=tool_definitions(tools))
480     agent = Agent(brain=brain, tools=tools, memory=memory, mode=mode,
481                  ↪ brain_name=brain_name)
482
483     print(f"⚡ Nanocode v0.6")
484     print(f"Commands: /q quit, /switch toggle brain, /mode [plan|act]")
485     print(f"Brain: {brain_name}")
486     if mode == "act":
487         print("Mode: ACT (Writing Enabled)")
488     else:
489         print("Mode: PLAN (Code Read-Only)")
490
491     while True:
492         try:
493             user_input = input(f"[{agent.brain_name}]:[{agent.mode}] > ")
494             output = agent.handle_input(user_input)
495             if output:
496                 print(f"\n{output}\n")
497
498             except (AgentStop, KeyboardInterrupt):
499                 print("\nExiting...")
500                 break
501
502 if __name__ == "__main__":
503     main()

```

Line 475: Parse --act from the command line. Default is "plan" (safe). **Line 492:** Show current brain and mode in the prompt: [claude:plan] >.

Testing the Harness

Run it:

```

1 python nanocode.py
2 ⚡ Nanocode v0.6
3
4 [claude:plan] > Write a hello world script called hello.py
5
6 [Tool Use: write_plan]
7 [Tool Output: Plan saved to PLAN.md...]
8
9 I've created a plan for the hello.py script in PLAN.md.
10
11 [claude:plan] > /mode act
12 ⚠ Switched to ACT MODE (Writing Enabled)
13
14 [claude:act] > Follow the plan
15
16 [Tool Use: write_file]
17 [Tool Output: Successfully wrote 47 characters to hello.py...]
18
19 Done! I've created hello.py with a simple hello world script.

```

Claude didn't try to write `hello.py` directly—it couldn't, because `write_file` wasn't on its menu. It used `write_plan` instead. After we switched to act mode, the full toolset became available and the write went through.

Or start directly in act mode:

```

1 python nanocode.py --act
2 ⚡ Nanocode v0.6
3 Commands: /q quit, /switch toggle brain, /mode [plan|act]
4 Brain: claude
5 Mode: ACT (Writing Enabled)
6
7 [claude:act] > ...

```

The Psychology of the “Plan”

There's a subtler benefit here, beyond safety.

When you force the agent to write `PLAN.md` first, you are forcing it to “think before it acts.” It reads your code, drafts a plan in English, you correct it (“No, don't use Flask, use FastAPI”), and only then do you switch to Act Mode. By the time the agent starts writing code, it has a fully approved spec in its own memory—since `PLAN.md` is just a file it can read.

Dev Tip: Good developers plan before they code. This feature forces the agent to do the same. Never let it code without a blueprint.

Dev Tip: We hide tools from the menu. Production agents like Claude Code take a different approach: all tools stay visible, but a permission gate returns allow, deny, or ask-the-user before each execution. That's more flexible—the agent can *propose* a write and get explicit approval—but it also means the LLM burns tokens generating tool calls that get rejected. Our menu-filtering approach is simpler and cheaper: the LLM can't misuse what it can't see.

Wrapping Up

The safety harness is two tool lists and a method that swaps between them. No blocking, no round trips, no wasted tokens on rejected tool calls. The LLM can't misuse tools it doesn't have.

But the agent still can't *find* anything on its own. Ask it “Fix the bug in authentication” and it has no idea which file to open.

Chapter 8: The Context Pipeline (Map & Search)

Imagine trying to navigate a new city, but you can only see through a paper towel tube. You can see details perfectly if you are looking right at them, but you have no idea where you are.

This is the “Keyhole Problem.”

Right now, our agent has `ReadFile`. It can see the content, but it doesn’t know the context. If you ask “Fix the bug in the authentication logic,” it has no idea if that logic is in `auth.py`, `login.py`, or `utils/security.py`.

To fix this, we need a “Context Pipeline”—a sequence of tools that progressively narrow the search space. `list_files` shows the project structure, `search_codebase` finds where a string appears, and `read_file` loads the code. Wide to narrow.



Figure 9. Context Pipeline: `list_files` reveals the project structure, `search_codebase` finds the target file and line, `read_file` loads the content, then the agent can edit or plan.

The ListFiles Tool

The first thing any developer does in a new repo is run `ls -R` (or look at the file tree in VS Code).

We need a tool that returns the file structure. However, we must be careful. If we just list everything, we will clutter the context with garbage like `__pycache__`, `.git`, and `venv`.

The Test:

```

1  def test_list_files_returns_file_tree():
2      """Verify ListFiles returns a tree structure."""
3      with tempfile.TemporaryDirectory() as tmpdir:
4          os.makedirs(os.path.join(tmpdir, "src"))
5          with open(os.path.join(tmpdir, "README.md"), 'w') as f:
6              f.write("# Test")
7          with open(os.path.join(tmpdir, "src", "main.py"), 'w') as f:
8              f.write("print('hello')")
9
10         tool = ListFiles()
11         context = ToolContext()
12         result = tool.execute(context, path=tmpdir)
13
14         assert "README.md" in result
15         assert "src/" in result
16         assert "main.py" in result
17
18
19  def test_list_files_skips_git_and_pycache():
20      """Verify ListFiles skips .git and __pycache__ directories."""
21      with tempfile.TemporaryDirectory() as tmpdir:
22          os.makedirs(os.path.join(tmpdir, ".git"))
23          os.makedirs(os.path.join(tmpdir, "__pycache__"))
24          with open(os.path.join(tmpdir, ".git", "config"), 'w') as f:
25              f.write("[core]")
26          with open(os.path.join(tmpdir, "__pycache__", "cache.pyc"), 'w') as f:
27              f.write("bytecode")
28          with open(os.path.join(tmpdir, "main.py"), 'w') as f:
29              f.write("print('hello')")
30
31         tool = ListFiles()
32         context = ToolContext()
33         result = tool.execute(context, path=tmpdir)
34
35         assert "config" not in result # .git/config
36         assert "cache.pyc" not in result # __pycache__
37         assert "main.py" in result

```

The Code:

```

297 class ListFiles:
298     """Lists files in the project structure."""
299     name = "list_files"
300     plan_safe = True
301     description = "Lists all files in the project structure. Useful to
302         ↳ understand the project layout."
303     input_schema = {
304         "type": "object",
305         "properties": {
306             "path": {"type": "string", "description": "The root path (default
307                 ↳ '.')"
308         }
309     }
310
311     def execute(self, context, path="."):
312         print(f" → Listing {path}")
313         try:
314             file_list = []
315             for root, dirs, files in os.walk(path):
316                 dirs[:] = [d for d in dirs if d not in {".git", "__pycache__",
317                     ↳ "venv", ".nanocode"}]
318
319                 level = root.replace(path, '').count(os.sep)
320                 indent = ' ' * 4 * (level)
321                 file_list.append(f"{indent}{os.path.basename(root)}/")
322                 subindent = ' ' * 4 * (level + 1)
323                 for f in files:
324                     file_list.append(f"{subindent}{f}")
325
326             return "\n".join(file_list)
327         except Exception as e:
328             return f"Error listing files: {e}"

```

The Walkthrough:

- **Line 314:** Prune garbage directories in place—`.git`, `__pycache__`, `venv`, and `.nanocode`. Mutating `dirs[:]` tells `os.walk` to skip them entirely, including all their subdirectories.
- **Lines 316–321:** Build an indented tree structure showing the hierarchy.

The SearchCodebase Tool

Now, let's say the agent sees a file named `main.py` calling a function `calculate_tax()`, but `calculate_tax` isn't defined in `main.py`. The agent needs to find where it is defined.

We need a “Grep” tool.

We could use `git grep` via subprocess, but that requires git to be installed and initialized. In the spirit of “Zero Magic,” we will write a pure Python searcher.

The Test:

```

1  def test_search_codebase_finds_matches():
2      """Verify SearchCodebase finds text in files."""
3      with tempfile.TemporaryDirectory() as tmpdir:
4          with open(os.path.join(tmpdir, "test.py"), 'w') as f:
5              f.write("def hello_world():\n    print('hello')\n")
6
7          tool = SearchCodebase()
8          context = ToolContext()
9          result = tool.execute(context, query="hello_world", path=tmpdir)
10
11         assert "test.py" in result
12         assert "hello_world" in result
13         assert ":1:" in result # Line number
14
15
16  def test_search_codebase_case_insensitive():
17      """Verify SearchCodebase is case-insensitive."""
18      with tempfile.TemporaryDirectory() as tmpdir:
19          with open(os.path.join(tmpdir, "test.py"), 'w') as f:
20              f.write("class HelloWorld:\n    pass\n")
21
22          tool = SearchCodebase()
23          context = ToolContext()
24          result = tool.execute(context, query="helloworld", path=tmpdir)
25          assert "HelloWorld" in result

```

The Code:

```

328 class SearchCodebase:
329     """Searches for a string in all files."""
330     name = "search_codebase"
331     plan_safe = True
332     description = "Searches the entire codebase for a text string. Useful to
333     ↪ find where functions or variables are defined."
334     input_schema = {
335         "type": "object",
336         "properties": {
337             "query": {"type": "string", "description": "The string to search
338             ↪ for"},
339             "path": {"type": "string", "description": "The root path (default
340             ↪ '.')"}
341         },
342         "required": ["query"]
343     }
344
345 def execute(self, context, query, path="."):
346     print(f" → Searching for '{query}'")
347     results = []
348     try:
349         for root, dirs, files in os.walk(path):
350             dirs[:] = [d for d in dirs if d not in {".git", "__pycache__",
351             ↪ "venv", ".nanocode"}]
352
353             for file in files:
354                 file_path = os.path.join(root, file)
355                 try:
356                     with open(file_path, 'r', encoding='utf-8',
357                     ↪ errors='ignore') as f:
358                         for i, line in enumerate(f):
359                             if query.lower() in line.lower():
360                                 results.append(f"{file_path}:{i+1}:
361                                 ↪ {line.strip()}")
362
363                     except Exception:
364                         continue
365
366         return "\n".join(results) if results else "No matches found."
367     except Exception as e:
368         return f"Error searching: {e}"

```

The Walkthrough:

- **Line 347:** Same `dirs[:]` pruning as `ListFiles`.
- **Lines 354-355:** Case-insensitive search, formatted as `file:line: content`.
- **Line 359:** Return a helpful message if nothing matches.

Update the Tools List

Add the new tools to the list:

```
402 tools = [ReadFile(), WritePlan(), SaveMemory(), ListFiles(), SearchCodebase(),  
    ↪ WriteFile()]
```

No other changes needed—the agentic loop already handles any tool by name.

The “Zoom In” Test

We’ll ask the agent to find something without telling it where to look.

Scenario: “Where is the Claude class defined?”

```
1 python nanocode.py  
2 ⚡ Nanocode v0.7  
3  
4 [claude:plan] › Where is the Claude class defined?  
5  
6 [Tool Use: search_codebase]  
7 [Tool Output: ./nanocode.py:144: class Claude(Brain):]  
8  
9 The `Claude` class is defined in `nanocode.py` on line 144.
```

The agent didn’t guess. It searched, found the line, and reported the fact. No hallucination. No “I think it might be in...”—just facts.



Aside: If you run from the project root, `search_codebase` will find all matches across your entire codebase—which is exactly what you want when working on real projects.

Wait, is this RAG?

You might have heard of RAG (Retrieval Augmented Generation) and expected us to install a Vector Database like Chroma or Pinecone.

What we just built is RAG—but the agentic kind.



Figure 10. Comparison of Vector RAG (passive, embedding-based) vs Agentic RAG (active, tool-based). Agentic RAG lets the LLM decide when and how to search.

Feature	Vector RAG (Passive)	Agentic RAG (Active)
Mechanism	Embeddings (semantic similarity)	Tools (explicit search)
Trigger	Automatic (every query)	On-demand (agent decides)
Precision	Semantic (finds related concepts)	Exact (finds literal matches)
Best For	Documentation, Q&A, semantic search	Debugging, code navigation

For a coding agent, agentic RAG is often the better fit. When debugging, you don’t want “semantically similar” code; you want the exact line where `calculate_tax` is defined. Vector search excels at conceptual queries (“What’s our refund policy?”), but keyword search excels at finding specific identifiers—and that’s most of what a coding agent does.

Dev Tip: `list_files` is cheap. `search_codebase` is cheap. `read_file` is expensive. By searching first, the agent avoids reading unnecessary files, saving you money on tokens.

Wrapping Up

Two tools, no dependencies, no vector database.

The agent is getting capable. But it still can’t verify its own work—it writes code and hopes for the best. We need to give it a terminal.



Aside: The original RAG paper by Lewis et al.¹ introduced the concept of augmenting generation with retrieval. Anthropic’s tool use documentation² covers the agentic approach we used here.

¹<https://arxiv.org/abs/2005.11401>

²<https://docs.anthropic.com/en/docs/build-with-claude/tool-use/overview>

Chapter 9: The Reality Check (Running Code)

There is a difference between writing code and knowing it works.

Right now, our agent can write a Python script. It can read existing code. It can search for function definitions. But it has no way to *verify* anything. It's a chef who has never tasted their own cooking. It might write `import pandas` without knowing if pandas is installed. It might write `def calculate_tax()` with a syntax error that looks plausible but crashes immediately.

The difference between generating code and writing *working* code is whether you bother to run it.

The Feedback Loop

To fix this, we need to close the *feedback loop*: the agent creates a script, executes it, reads the error message, edits the file, and runs it again—over and over until the tests pass.

This is how humans code. You write something, run it, see the traceback, fix it, run it again. The terminal is your teacher.



Warning: Giving an AI access to your shell is the most dangerous thing we've done so far. `rm -rf /` is a valid shell command. This tool *must* stay off the plan mode menu.

Dev Tip: The dangers are subtler than `rm -rf`. A command like `timeout 5 rm -rf /` looks safe if you only inspect the first word, but it's `rm` in disguise. One useful first defense: strip wrapper programs (`timeout`, `nice`, `nohup`) before analyzing what's actually being executed. This doesn't close the shell-safety

problem—separators, subshells, and expansion are separate hazards—but it catches a common class of indirection.

Tests First

We define the behavior before writing the implementation:

```

1  def test_run_command_executes():
2      """Verify run_command executes shell commands."""
3      tool = RunCommand()
4      context = ToolContext()
5      result = tool.execute(context, command="echo hello")
6
7      assert "hello" in result
8
9
10 def test_run_command_captures_stderr():
11     """Verify run_command captures error output."""
12     tool = RunCommand()
13     context = ToolContext()
14     result = tool.execute(context, command="python -c \"import sys;
15         ↪ sys.stderr.write('error!')\"")
16
17     assert "STDERR" in result
18     assert "error!" in result
19
20 def test_run_command_timeout(monkeypatch):
21     """Verify run_command times out on long-running commands."""
22     monkeypatch.setenv("NANOCODE_TIMEOUT", "1")
23     tool = RunCommand()
24     context = ToolContext()
25     result = tool.execute(context, command="sleep 100")
26
27     assert "timed out" in result

```

The RunCommand Tool

We need a tool that wraps Python's subprocess module:

```

423 class RunCommand:
424     """Executes shell commands."""
425     name = "run_command"
426     plan_safe = False
427     description = "Executes a terminal command. Use this to run scripts, tests,
↳ or install packages."
428     input_schema = {
429         "type": "object",
430         "properties": {
431             "command": {"type": "string", "description": "The shell command to
↳ run (e.g., 'python test.py')"}
432         },
433         "required": ["command"]
434     }
435
436     def execute(self, context, command):
437         print(f" → Running: {command[:50]}...")
438         try:
439             result = subprocess.run(
440                 command,
441                 shell=True,
442                 capture_output=True,
443                 text=True,
444                 timeout=int(os.environ.get("NANOCODE_TIMEOUT", "30")),
445                 cwd=os.getcwd()
446             )
447
448             output = ""
449             if result.stdout:
450                 output += f"STDOUT:\n{result.stdout}\n"
451             if result.stderr:
452                 output += f"STDERR:\n{result.stderr}\n"
453             if not output:
454                 output = "(No output)"
455
456             return output.strip()
457
458         except subprocess.TimeoutExpired:
459             return "Error: Command timed out."
460         except Exception as e:
461             return f"Error executing command: {e}"

```

The Walkthrough:

- **Lines 439-446:** `subprocess.run()` with a configurable timeout (see Dev Tip below).
- **Lines 449-454:** We include both `stdout` and `stderr` so Claude can see error messages.

The safety guardrail isn't in the tool—it's in the menu. `RunCommand` is an act-only tool, so it never appears on the brain's menu in plan mode. The LLM can't call what it can't see.



Warning: Add `import subprocess` at the top of `nanocode.py` before proceeding. Without it, `RunCommand` will crash with `NameError`.

Add `RunCommand()` to the act-only tools:

```
tools = [ReadFile(), WritePlan(), SaveMemory(), ListFiles(), SearchCodebase(),  
↪ WriteFile(), RunCommand()]
```

The Interactive Trap

Dev Tip: The default timeout is 30 seconds, configurable via `NANOCODE_TIMEOUT=300` for long-running tasks. Why have one at all? If the agent runs `python` (entering the REPL) or `npm init` (waiting for user input), the script would hang forever. Commands that require interactive input will timeout instead of blocking your session. Never give an agent a shell without a timeout.

There's a related gotcha: some commands like `git log` or `less` use a pager that waits for user input. The agent needs to use flags like `git --no-pager log` or pipe to head. Over time, the agent will learn these patterns through trial and error—which is exactly the point of giving it a feedback loop.

The Self-Healing Demo

We're going to intentionally ask the agent to write broken code and watch it fix itself.

Scenario: "Write a Python script that divides by zero, run it, and fix the error."

```

1  python nanocode.py --act
2  ⚡ Nanocode v0.8
3  Mode: ACT (Writing Enabled)
4
5  [claude:act] > Write a script bug.py that divides 10 by 0. Run it. Fix it.
6
7      → Writing bug.py
8
9      → Running: python bug.py...
10  STDERR:
11  Traceback (most recent call last):
12      File "bug.py", line 1, in <module>
13          print(10 / 0)
14  ZeroDivisionError: division by zero
15
16  I see the ZeroDivisionError. Let me fix the script to handle this case.
17
18      → Writing bug.py
19
20      → Running: python bug.py...
21  STDOUT: Cannot divide by zero!
22
23  Done. The script now handles the division by zero gracefully.

```

You didn't do anything except watch.

The TDD Workflow

With `run_command`, you can now use a professional Test-Driven Development workflow:

```

1  [claude:act] > Run the tests for this project.
2
3      → Running: pytest...
4  [Tool Output:
5  STDOUT:
6  ===== test session starts =====
7  ...
8  FAILED test_calculator.py::test_divide - ZeroDivisionError
9  ===== 1 failed, 3 passed =====
10 ]
11
12 I see one failing test. Let me investigate...
13
14 [Tool Use: read_file(path='test_calculator.py')]

```

```

15 ...
16
17 [Tool Use: read_file(path='calculator.py')]
18 ...
19
20 I found the bug. The divide function doesn't handle zero denominators.
21
22 [Tool Use: write_file(path='calculator.py', content='...')]
23 [Tool Output: Successfully wrote 156 characters to calculator.py]
24
25 [Tool Use: run_command(command='pytest')]
26 [Tool Output:
27 STDOUT:
28 ===== test session starts =====
29 ===== 4 passed =====
30 ]
31
32 All tests pass now.

```

Run tests, read errors, fix code, verify.

The Surgical Edit

Look at the TDD workflow above:

```

1 [Tool Use: write_file(path='calculator.py', content='...')]

```

To fix one bug, the agent rewrote the entire file. For a small fix, this is wasteful—and for a large file, it's a problem. What if `calculator.py` had 500 lines? The agent would need to output all 500 lines just to change one.

We need a surgical tool.

The Test:

```

1  def test_edit_file_replaces_text():
2      """Verify EditFile replaces text in a file."""
3      with tempfile.NamedTemporaryFile(mode='w', suffix='.py', delete=False) as
         ↪ f:
4          f.write("x = 1\ny = 2\nz = 3\n")
5          temp_path = f.name
6
7      try:
8          tool = EditFile()
9          context = ToolContext()
10         result = tool.execute(context, temp_path, "y = 2", "y = 42")
11
12         assert "Successfully" in result
13         with open(temp_path) as f:
14             content = f.read()
15             assert "y = 42" in content
16             assert "y = 2" not in content
17     finally:
18         os.unlink(temp_path)

```

The Code:

```

305 class EditFile:
306     """Replaces text in a file (surgical edit)."""
307     name = "edit_file"
308     plan_safe = False
309     description = "Replaces specific text in a file. Use for surgical edits
         ↪ instead of rewriting entire files."
310     input_schema = {
311         "type": "object",
312         "properties": {
313             "path": {"type": "string", "description": "Path to the file"},
314             "old_text": {"type": "string", "description": "Exact text to find
                 ↪ and replace"},
315             "new_text": {"type": "string", "description": "Text to replace it
                 ↪ with"}
316         },
317         "required": ["path", "old_text", "new_text"]
318     }
319
320     def execute(self, context, path, old_text, new_text):
321         print(f" → Editing {path}")
322         try:
323             with open(path, 'r', encoding='utf-8') as f:
324                 content = f.read()
325                 if old_text not in content:
326                     return f"Error: Could not find the specified text in {path}"
327                 new_content = content.replace(old_text, new_text, 1)

```

```
328         with open(path, 'w', encoding='utf-8') as f:
329             f.write(new_content)
330         return f"Successfully edited {path}"
331     except Exception as e:
332         return f"Error editing file: {e}"
```

The Walkthrough:

- **Line 325:** We check if the old text exists. If not, return an error—this catches typos and misremembered code.
- **Line 327:** `replace(old_text, new_text, 1)` replaces only the first occurrence. This is intentional—if you want to replace all occurrences, call it multiple times or use a more specific match.

Like `RunCommand`, `EditFile` is an act-only tool—it's not on the brain's menu in plan mode. Add it to the tools list:

```
tools = [ReadFile(), WritePlan(), SaveMemory(), ListFiles(), SearchCodebase(),
→ WriteFile(), EditFile(), RunCommand()]
```

Now the agent can fix bugs with precision. Instead of rewriting 500 lines, it replaces just the broken part.

The Closed Loop

`run_command` opens up more than testing. `curl`, `git`, `sqlite3`—any CLI tool on your system is now available to the agent. We didn't implement MCP or any protocol. We just gave it a shell.

Hardening the Loop

The feedback loop works, but it has two problems: conversations grow without bound, and a hallucinating model can loop forever. Both need fixing before we trust this in the wild.

Context Compaction

With feedback loops, a single user request can trigger 20+ API calls as the agent writes code, runs tests, reads errors, and fixes them. Each round appends messages—tool calls, file contents, error output—to the conversation list. It grows fast. A stubborn bug that takes 15 rounds of write-run-fix can easily balloon the conversation past 100,000 tokens. Eventually you hit the context limit we discussed in Chapter 3, and the API rejects your request entirely.

Every provider tells you exactly how close you are. The response JSON includes a `usage` field with `input_tokens`—the total tokens consumed by the conversation you just sent. We can track this after each API call:

Dev Tip: While we're tracking tokens for compaction, you can also track spend. Add a running total—`self.total_input_tokens += data["usage"]["input_tokens"]`—and print it on exit. At Claude's current pricing, rough cost is `total_input_tokens / 1_000_000 * 3` dollars. Useful for catching runaway loops before they become painful invoices.

```
183         response = request_with_retry(self.url, headers, payload)
184         data = response.json()
185         self.last_input_tokens = data.get("usage", {}).get("input_tokens", 0)
186         return self._parse_response(data["content"])
```

We also need each Brain subclass to declare its limit. Claude allows 200,000 tokens, DeepSeek allows 128,000. These go on the base class as defaults that subclasses can override:

```

115 class Brain:
116     """Base class for LLM providers."""
117     context_limit = 200_000 # Subclasses override as needed
118     last_input_tokens = 0   # Updated after each think() call
119
120     def think(self, conversation):
121         """Process conversation, return Thought."""
122         raise NotImplementedError

```

DeepSeek overrides the limit and tracks tokens the same way:

```

189 class DeepSeek(Brain):
190     """DeepSeek API (Anthropic-compatible, with tool support)."""
191     context_limit = 128_000

```

Apply the same `last_input_tokens` tracking to DeepSeek's `think()` method—both brains need to report their token usage for compaction to work:

```

    data = response.json()
    self.last_input_tokens = data.get("usage", {}).get("input_tokens", 0)
    return self._parse_response(data["content"])

```

Now in `_agentic_loop`, we check after each API call whether we've passed 75% of the limit. If so, we summarize and replace the history. Note the timing: compaction happens after `think()` returns but before the response is appended. The current response's tool calls still execute normally against the now-summarized history—a minor discontinuity, but the alternative (crashing against the context limit) is worse.

```

558         thought = self.brain.think(self.conversation)
559
560         # Display thinking
561         if thought.thinking:
562             lines = thought.thinking.strip().split("\n")[:5]
563             for i, line in enumerate(lines):
564                 prefix = "  🗨 " if i == 0 else "    "
565                 print(f"\033[2m{prefix}{line}\033[0m")
566
567         # Compact if approaching context limit
568         if self.brain.last_input_tokens > self.brain.context_limit * 0.75:
569             self._compact_conversation()

```

The compaction method itself is simple—serialize the conversation, ask the brain to summarize, and replace the old messages:

```

598     def _compact_conversation(self):
599         """Summarize old messages to stay within context limits."""
600         print("(Compacting conversation...)")
601         history = "\n".join(
602             f"{m['role']}: {str(m['content'])[:500]}"
603             for m in self.conversation
604         )
605         prompt = [{
606             "role": "user",
607             "content": f"Summarize this conversation for continuity. "
608                       f"Focus on what was accomplished, what's in progress, "
609                       f"and key decisions:\n\n{history}"
610         }]
611         saved_tools = self.brain.tools
612         self.brain.tools = []
613         try:
614             thought = self.brain.think(prompt)
615         finally:
616             self.brain.tools = saved_tools
617         self.conversation = [
618             {"role": "user", "content": f"Previous conversation summary: "
619                                         ↪ f"{thought.text}"},
619     ]

```

Notice `saved_tools`: we strip the tool menu before calling `think()` for the summary, then restore it. Without this, the model might try to use tools instead of writing a summary. The `try/finally` ensures tools are restored even if the summarization call fails.

We replace the entire history with a single user message. The current assistant response gets appended by the normal loop flow on the next line, maintaining proper user/assistant alternation.

While we’re hardening the loop, let’s also add the iteration guard we’ve been deferring since Chapter 5. We replace the `while True` loop with a bounded `for` loop. With `run_command` in the mix, a hallucinating model can burn real money:

```
def _agentic_loop(self):
    """Process brain responses, executing tools until done."""
    output_parts = []
    max_iterations = 50

    for _iteration in range(max_iterations):
        thought = self.brain.think(self.conversation)
        # ... compaction, tool execution, etc.
        if not thought.tool_calls:
            break
    else:
        output_parts.append("(Stopped: too many iterations)")

    return "\n".join(output_parts)
```

The `for/else` pattern is a clean way to cap iterations—the `else` clause fires only if the loop exhausted all iterations without hitting `break`. Fifty rounds is generous enough for most tasks but prevents runaway loops from draining your API budget.

Summarization loses detail. The brain won’t remember exactly which line had the bug or the precise error message from three iterations ago. But a crashed agent—one that hit the context limit and got rejected—loses everything. The summary preserves the arc of what happened; the details are in the files the agent already wrote to disk.

Two failure modes to watch for. First, *summary drift*: each compaction summarizes the previous summary, and small inaccuracies compound. After three compactions, the agent might “remember” fixing a bug it never actually fixed. Second, *tool-result loss*: file contents and error messages get compressed into vague descriptions like “there was a `TypeError`.” Both are acceptable trade-offs—a crashed agent is worse than a forgetful one—but if you see the agent going in circles, restart the conversation. The files on disk are the real source of truth.

Dev Tip: Before reaching for expensive summarization, try the cheap thing first: clear old tool results. A `read_file` output from 20 turns ago is already baked into the LLM’s understanding—keeping the full text just wastes tokens. A common production pattern is “microcompaction”—replacing stale tool results with a short placeholder and only triggering full summarization if that isn’t enough.

Security Considerations



Aside: We’re running shell commands with `shell=True`, which has security implications. If the agent constructs commands from untrusted input, it could be vulnerable to shell injection. For our use case—an interactive coding assistant—this is acceptable because the agent is executing commands on your behalf in your development environment. But you wouldn’t deploy this to a production server accepting user input.

For additional safety, consider:

- Running in a Docker container or VM
- Using a restricted shell or sandboxed environment
- Adding a command allowlist (only `python`, `pytest`, `pip`, etc.)
- Requiring confirmation before executing each command

Plan mode handles the common case—it prevents accidental file modifications during exploration. But in act mode, `shell=True` gives full shell access. If you use act mode regularly, running inside a container is cheap insurance, not just a production concern.

Dev Tip: You’ve seen `except Exception` throughout our tools. This is a deliberate simplification—a CLI agent should never crash from a bad file path or a network hiccup. But in production, narrow your catches. `ReadFile` benefits from `except FileNotFoundError` and `except PermissionError` before the

generic fallback—the agent gets a more useful error message, and you don’t accidentally swallow bugs. We keep the broad catch as a safety net, not as the first line of defense.

Wrapping Up

The loop is closed, and compaction keeps it from blowing the context window. But there’s still the bill—each iteration hits the API. A stubborn bug could trigger 30 rounds of write-run-fix, and you’re paying for every one. Time to sever the cord.

Part III: The Frontier

The core agent is complete. Now you'll push it further: run it on local models for free, add web search so it can learn on the fly, and put everything to the test in a capstone where the agent builds a complete Snake game from scratch.

Chapter 10: Going Dark (Local Models)

We have a problem.

The agent can run code now. This enables *iteration*—the agent tries, fails, reads the error, and tries again. But each iteration costs money. If the agent loops 50 times debugging a stubborn syntax error, and you’re paying \$0.03 per turn, you wake up to a painful credit card bill.

This is “Taxi Meter Anxiety”: the fear of letting an autonomous system run unsupervised because the meter is always ticking.

To build an autonomous agent—one you can leave running overnight—we need to sever the cord. We need to go *local*.

The Trade-off

The numbers tell the story:

Provider	SWE-bench	Cost	Latency
Claude Sonnet 4.6	~72% (early 2026)	\$3/\$15 per MTok (in/out)	1-3 seconds
Qwen3-Coder-30B (local)	~52%	Free	Depends on hardware

SWE-bench tests models on real GitHub issues—understanding codebases, implementing fixes, passing tests.¹ Claude scores significantly higher, but the local model is free. For debugging loops where the agent is just fixing typos and running `python script.py` over and over, “free” wins. These numbers shift fast—check the leaderboard for current rankings by the time you read this.

¹<https://www.swebench.com>

Installing Ollama

Ollama is a tool that lets you run open-source LLMs on your own laptop. It handles model downloading and serving both OpenAI-compatible and Anthropic-compatible APIs. We'll use the Anthropic-compatible endpoint so all three brains share the same message format.

On macOS (Homebrew):

```
1 brew install ollama
2 brew services start ollama
```

On Linux:

```
1 curl -fsSL https://ollama.com/install.sh | sh
```



Warning: Piping a remote script into `sh` runs arbitrary code with your user's permissions. Review the script first (`curl -fsSL https://ollama.com/install.sh | less`) or use your distro's package manager if Ollama is available there.

Pull a coding model:



Warning: `qwen3-coder:30b` needs approximately 24GB of RAM/VRAM to run smoothly. If your machine has less than 16GB of RAM, use a smaller model instead: `ollama pull llama3.2:3b` (2GB, fast but less capable) or `ollama pull deepseek-coder:6.7b` (4GB, better at code). You can always switch models later via the `OLLAMA_MODEL` environment variable.

```
1 ollama pull qwen3-coder:30b
```

`qwen3-coder:30b` is an open-source coding model from Alibaba's Qwen team—one of the best open-source alternatives for code generation as of this writing.

Verify it works:

```
1 ollama run qwen3-coder:30b "Write a Python hello world"
```

You should see generated Python code. If you do, your local brain is ready.

Dev Tip: The first run downloads several gigabytes. `qwen3-coder:30b` is approximately 19GB (varies by quantization). Make sure you have disk space and a decent internet connection for the initial pull.



Warning: Ollama's Anthropic-compatible API requires version 0.6.0 or newer. Run `ollama --version` to check, and update if you see 404 errors on the `/v1/messages` endpoint.

The Ollama Brain Class

Ollama now provides an Anthropic-compatible API, just like DeepSeek. All three brains—Claude, DeepSeek, and Ollama—use the same message format and support tool calling.

```
225 class Ollama(Brain):
226     """Ollama local model (Anthropic-compatible API, with tool support)."""
227     context_limit = 32_000
228
229     def __init__(self, memory=None, tools=None):
230         self.memory = memory
231         self.tools = tools or []
232         self.system = None
233         self.model = os.getenv("OLLAMA_MODEL", "qwen3-coder:30b")
234         self.url = "http://localhost:11434/v1/messages"
235         self._detect_context_limit()
236
237     def _detect_context_limit(self):
238         """Query Ollama for the model's context window size."""
239         try:
240             response = requests.post(
241                 "http://localhost:11434/api/show",
242                 json={"model": self.model},
243                 timeout=2
```

```

244         )
245         model_info = response.json().get("model_info", {})
246         for key, value in model_info.items():
247             if key.endswith(".context_length"):
248                 self.context_limit = value
249             return
250     except Exception:
251         pass # Keep default
252
253     def think(self, conversation):
254         headers = {
255             "x-api-key": "ollama", # Required but not validated
256             "anthropic-version": "2023-06-01",
257             "content-type": "application/json"
258         }
259         payload = {
260             "model": self.model,
261             "max_tokens": 4096,
262             "messages": conversation
263         }
264         if self.system:
265             payload["system"] = self.system
266         if self.tools:
267             payload["tools"] = self.tools
268
269         response = request_with_retry(self.url, headers, payload)
270         data = response.json()
271         self.last_input_tokens = data.get("usage", {}).get("input_tokens", 0)
272         return self.parse_response(data["content"])

```

The Walkthrough:

- **Line 227:** `context_limit = 32_000` is the class-level default, used when `_detect_context_limit()` can't query Ollama. This feeds the context compaction from Chapter 9.
- **Lines 229-235:** No API key required—Ollama runs locally. The model defaults to `qwen3-coder:30b` but can be overridden via `OLLAMA_MODEL`. Ollama serves an Anthropic-compatible endpoint on port 11434. The last line calls `_detect_context_limit()` to replace the hardcoded default with the model's actual limit.
- **Lines 237-251:** Ollama's `/api/show` endpoint returns model metadata, including the context window size under a key like `qwen3.context_length` in the `model_info` dict. We iterate the keys looking for one ending in `.context_length`. If Ollama isn't running or the model isn't pulled, the `except` catches the error and keeps the default. The rest of the class

is identical in structure to `Claude` and `DeepSeek—_parse_response` is inherited from the `Brain` base class.

Add `Ollama` to the `BRAINS` registry:

```
275 # Available brains
276 BRAINS = {
277     "claude": Claude,
278     "deepseek": DeepSeek,
279     "ollama": Ollama,
280 }
```

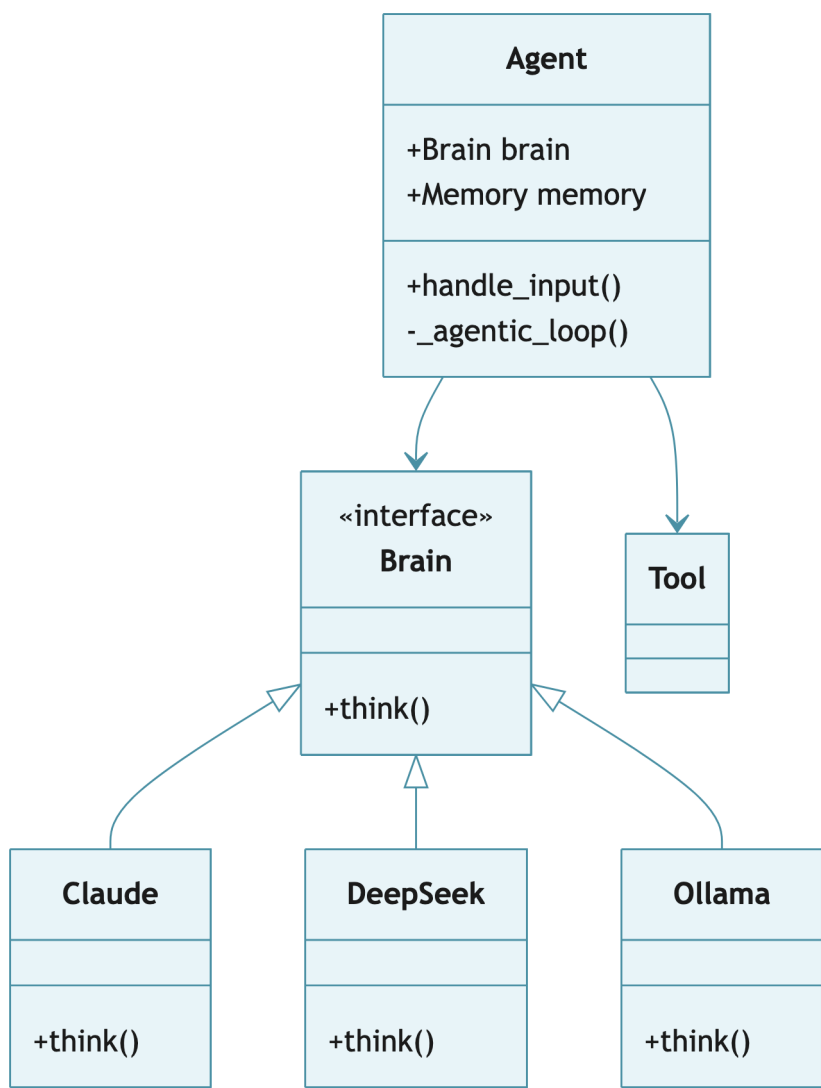


Figure 11. Class diagram showing all three Brain implementations: Claude, DeepSeek, and Ollama all implement the Brain interface.

Running with Ollama

Set the environment variable and start the agent:

```

1 export NANOCODE_BRAIN=ollama
2 python nanocode.py --act

```

```

1 ⚡ Nanocode v0.9
2 Commands: /q quit, /switch toggle brain, /mode [plan|act]
3 Brain: ollama
4 Mode: ACT (Writing Enabled)
5
6 [ollama:act] > Write a Python script that prints the Fibonacci sequence up to
  ↳ 100.
7
8 Here's a Python script that prints the Fibonacci sequence up to 100:
9
10     a, b = 0, 1
11     while a <= 100:
12         print(a)
13         a, b = b, a + b

```

The response came from your laptop, not from Anthropic’s servers. No API key. No bill.

The “Infinite Loop” Experiment

Now we can run experiments that would be too expensive on Claude. Let’s try something ambitious:

```

1 [ollama:act] > Write a Python script 'mandelbrot.py' that renders the
  ↳ Mandelbrot set
2 in ASCII art. Run it. If it doesn't look good, tweak the character set and try
  ↳ again.

```

Watch the terminal. You’ll see the agent loop:

1. Writes code
2. Runs it via `run_command`
3. Sees the output
4. Decides it looks messy
5. Rewrites with different characters
6. Runs it again

It might take 10 iterations. It might take 30. It costs you nothing except electricity.

The Practical Differences

Local models like `qwen3-coder:30b` are excellent for straightforward coding tasks. For complex reasoning and agentic workflows, cloud models like Claude still have an edge—the 30B model you run locally scores ~52% on SWE-bench compared to Claude’s ~72%. That gap matters for complex multi-file refactors, but not for fixing syntax errors.

Speed depends on your hardware. With a good GPU (or Apple Silicon), responses come in seconds. On CPU, expect 10–30 seconds per response. The model isn’t “thinking harder”—it’s just running on slower hardware.

Instruction-following also varies. Cloud models like Claude are heavily fine-tuned for chat and instruction-following. They’re trained to infer what you *meant*, not just what you *said*. Local models may need more explicit instructions.

When working with local models, clearer prompts help:

- Be specific about what you want
- Break tasks into smaller steps
- Provide examples when possible

This is good practice for *any* model, but it matters more when you’re not relying on heavy instruction-tuning to fill in the gaps.

You might also notice identity confusion—ask `qwen3-coder:30b` what model it is, and it might say “I’m Claude” before correcting itself. Local models are trained on internet data that includes other models’ outputs, so they sometimes get confused about who they are. It’s harmless but amusing.



Aside: This is actually good practice. If you can get a local model to do the task, Claude will do it even better. Prompts that work on weak models work even better on strong models.

The Hybrid Workflow

Smart developers use a “Sandwich Workflow”:

Start with Claude to write `PLAN.md`—let the smart model make architectural decisions. Then switch to Ollama to grind out the implementation, iterating on syntax errors and running tests for free. When the code works, switch back to Claude to audit the result and catch logical errors.

You can switch brains mid-session with `/switch`:

```
1 [claude:plan] > Analyze this codebase and write a plan to add user
  ↳ authentication.
2 (Claude is thinking...)
3 [writes detailed PLAN.md]
4
5 [claude:plan] > /switch
6 Switched to: deepseek
7
8 [deepseek:plan] > /switch
9 Switched to: ollama
10
11 [ollama:plan] > /mode act
12 ⚠ Switched to ACT MODE (Writing Enabled)
13
14 [ollama:act] > Implement step 1 from PLAN.md
15 [grinds through implementation]
```

Use Claude for tasks that need strong reasoning, and Ollama for repetitive or latency-tolerant work where you’d rather not pay per token.

Model Selection

Different local models have different strengths (as of early 2026—check Ollama’s docs for current recommendations):

Model	Size	Best For
qwen3-coder:30b	~19GB	General coding, Python
codellama:7b	~4GB	Code completion
deepseek-coder:6.7b	~4GB	Complex reasoning
llama3.2:3b	~2GB	Fast responses, simple tasks

You can switch models by setting `OLLAMA_MODEL`:

```
1 export OLLAMA_MODEL=llama3.2:3b
2 python nanocode.py --act
```

Experiment to find the sweet spot for your hardware.



Warning: Local models run on your CPU/GPU. The 30B model needs 24GB+ RAM/VRAM; smaller models (3B-7B) run on 8GB. If you have less than 24GB, stick with the smaller models from the table above. On machines without a dedicated GPU, expect responses to take 10-30 seconds regardless of model size.

Troubleshooting Ollama

Error	Cause	Fix
Connection refused	Ollama not running	Run <code>ollama serve</code> or <code>brew services start ollama</code>
Model not found	Model not pulled	Run <code>ollama pull qwen3-coder:30b</code>
Out of memory	Model too large	Try a smaller model: <code>ollama pull llama3.2:3b</code>
Very slow responses	Running on CPU	Normal for CPU. Consider a smaller model or GPU upgrade
Garbled JSON output	Model confusion	Some models struggle with tool calling. Try <code>qwen3-coder</code> which handles it well

If Ollama seems stuck, check its logs with `ollama logs` or restart the service.

Wrapping Up

The Adapter Pattern paid off. Adding Ollama took about 50 lines of code and zero changes to the Agent class. All three brains use the same message format and the same `think()` interface.

Those feedback loops from Chapter 9 are now free. Let the agent iterate 50 times on a syntax error—you're paying in electricity, not API credits.

But cheaper models are also staler models. Ollama runs whatever weights you've downloaded—it has no knowledge of anything that happened after its training cutoff. When the agent needs current information, it's stuck. Time to give it the internet.

Chapter 11: The Extension (Web Search)

The agent can read, write, search, and run code. But it is trapped in its training data.

Ask it “What is the latest version of Next.js?” and it will hallucinate or say “I don’t know, my training data cuts off.” The traditional fix is to wait for Anthropic to release a new model.

We could also just give it the internet.

We won’t write the code ourselves. Instead, we’ll hand the keyboard to the agent. The tool pattern from Chapter 5 is simple enough that an LLM can follow it. We’ll show the reference implementation afterward for readers who prefer to type it themselves.

We’ll ask Nanocode to open `nanocode.py`, implement a web search tool, and wire it into its own brain.

Before starting, commit your current code so you can diff against the agent’s changes: `git add -A && git commit -m 'pre-ch11'`. If the edit goes sideways, you can always restore from the book’s code: `cp resources/code/ch11/nanocode.py .`

Step 1: The Meta-Prompt

Start your agent in act mode (so it has permission to write files):

```
1 python nanocode.py --act
```

Now, issue the command:

The Prompt:

- 1 Add a SearchWeb tool to nanocode.py that searches DuckDuckGo.
- 2 Follow the same pattern as other tools.

Step 2: The Surgery

Watch the terminal. The agent reads `nanocode.py` to understand the tool pattern, drafts the `SearchWeb` class following it, then uses `edit_file` to insert the new class and update the tools list.

If all goes well, you'll see:

- 1 → Reading `nanocode.py`
- 2 → Editing `nanocode.py`
- 3 → Editing `nanocode.py`



Aside: LLMs are non-deterministic. Your agent may produce slightly different code—perhaps using a different API or formatting results differently. That's fine. The reference implementation in `resources/code/ch11/` is the canonical version.

Once it finishes, run `git diff nanocode.py` to inspect what changed before restarting. The agent may have added imports, modified the tools list, or structured the code differently than you'd expect.



Warning: Self-modification is powerful but dangerous. Always review the diff before restarting. In production, you'd want version control, code review, and rollback capabilities. For a developer tool on your own machine, a feature branch and `git diff` are usually enough.



Warning: The agent can now edit its own source code *and* execute shell commands. In theory, it could disable plan mode by editing `nanocode.py`, then run anything it wants. For a local developer tool this is acceptable—you're reviewing diffs and running in your own environment. For anything shared or production-facing, run the agent in a Docker container or VM where self-modification can't escape the sandbox.

Step 3: The Reference Implementation

Since LLM output varies, here's the canonical version. First, install the dependency (this is the one addition to our stack beyond requests and python-dotenv):

```
1 pip install ddgs
```

The SearchWeb class checks whether the library loaded. Add the import at the top of nanocode.py, alongside the other imports:

```
7 try:
8     from ddgs import DDGS
9 except ImportError:
10     DDGS = None
```

If you haven't installed ddgs, the agent will still start—SearchWeb.execute() returns a helpful error instead of crashing.

The SearchWeb class:

```
523 class SearchWeb:
524     """Searches the internet using DuckDuckGo."""
525     name = "search_web"
526     plan_safe = True
527     description = "Searches the internet for current information. Use when you
528     ↪ need knowledge beyond your training data."
529     input_schema = {
530         "type": "object",
531         "properties": {
532             "query": {"type": "string", "description": "The search query"}
533         },
534         "required": ["query"]
535     }
536     def execute(self, context, query):
537         print(f" → Searching web for '{query}'")
538         if DDGS is None:
539             return "Error: ddgs package not installed. Run: pip install ddgs"
540         try:
541             results = DDGS().text(query, max_results=3)
```

```

542         if not results:
543             return "No results found."
544
545         formatted = []
546         for r in results:
547             formatted.append(f>Title: {r['title']}\nURL:
↪      {r['href']}\nSummary: {r['body']}\n")
548
549         return "\n".join(formatted)
550     except Exception as e:
551         return f"Error searching web: {e}"

```

SearchWeb is read-only (`plan_safe = True`), so it's available even in plan mode:

```

571 tools = [ReadFile(), WritePlan(), SaveMemory(), ListFiles(), SearchCodebase(),
↪      SearchWeb(),
572           WriteFile(), EditFile(), RunCommand()]

```

Step 4: The Tests

Add automated tests to `test_nanocode.py` to verify the new tool:

```

1  def test_search_web_tool_exists():
2      """Verify SearchWeb has required attributes."""
3      tool = SearchWeb()
4      assert tool.name == "search_web"
5      assert tool.description is not None
6      assert tool.input_schema is not None
7
8
9  def test_search_web_in_tools_list():
10     """Verify SearchWeb is in the tools list."""
11     assert any(t.name == "search_web" for t in tools)
12
13
14  def test_search_web_execute_success(monkeypatch):
15     """Verify SearchWeb.execute() returns formatted results."""
16     fake_results = [
17         {"title": "Python 3.13", "href": "https://python.org", "body": "Latest
↪      release"},
18     ]
19     monkeypatch.setattr("nanocode.DDGS", lambda: type("FakeDDGS", (), {

```

```

20         "text": lambda self, q, max_results=3: fake_results
21     })()
22     tool = SearchWeb()
23     context = ToolContext()
24     result = tool.execute(context, "latest python version")
25     assert "Python 3.13" in result

```

With web search wired in, Nanocode hits v1.0. Restart the agent and ask a question about the present:

```
1 python nanocode.py
```

```

1 ⚡ Nanocode v1.0
2 [claude:plan] > What is the latest stable version of Python?
3
4 → Searching web for 'latest stable Python version'
5
6 Based on search results, the latest stable version of Python is 3.13,
7 released in October 2024.

```

(This transcript was captured in October 2025. Your output will differ—the agent hits live search results.)

It worked. The agent realized it didn't know the answer, used the tool it just installed, and retrieved the information.

Dev Tip: Web search results are unverified. The agent may cite outdated pages or SEO spam as fact. For anything that matters—dependency versions, API behavior, security advisories—verify against official documentation. A good habit: tell the agent “search for X, then read the official docs page to confirm.”

If you see an error like `Error searching web: No module named 'ddgs'`, you need to install the `ddgs` package from Step 3.

Self-Modification

The agent used the same tools on its own source that it uses on your code—`read_file` to understand the architecture, `edit_file` to extend it.

In production, you wouldn't let an agent rewrite its core logic unsupervised. But for a developer tool on your own machine, you just skip the middleman. Need a new capability? Ask:

- “Add a tool to read PDF files.”
- “Add a tool to post to Slack.”
- “Refactor your memory system to use JSON instead of Markdown.”

Keep `git diff` close at hand, though.

Wrapping Up

The tool pattern from Chapter 5—`name`, `description`, `input_schema`, `execute()`—is simple enough that an LLM can follow it. The agent read the existing tools, matched the pattern, and wired in a new one. Every tool looks the same, so adding one is mechanical.

The agent has every tool it needs: `read`, `write`, `edit`, `search`, `run`, `remember`, and now `browse`. Time to put it all together on a real project.

Chapter 12: The Capstone (Building a Game)

Can Nanocode actually build something?

The “Zero Code Challenge”: build a classic Snake game using Python and Pygame. The rule—you are not allowed to write a single line of Python. You can only speak to the agent in English.



Aside: This demo involves many API calls. If you hit rate limits (HTTP 429), the agent will retry automatically. For long sessions, consider using a local model via Ollama to avoid limits entirely.

Step 1: Preparation

From your project root (the directory containing `nanocode.py` and `.env`), create a working directory for the game:



Aside: We work in a subdirectory so the agent’s `list_files` sees only the game files, not its own source code, and so it starts with a fresh memory.

```
1 mkdir -p snake_game
2 cp nanocode.py snake_game/
3 cp .env snake_game/
4 cd snake_game
```

If you’re using the book’s code repository, copy from `ch11` instead:

```
1 mkdir -p snake_game
2 cp resources/code/ch11/nanocode.py snake_game/
3 cp .env snake_game/
4 cd snake_game
```

Install Pygame:

```
1 pip install pygame
```



Aside: The shell commands above are for macOS/Linux. On Windows, use `mkdir snake_game`, `copy nanocode.py snake_game\`, and `copy .env snake_game\` instead. If `pip install pygame` fails on macOS, you may need `brew install sdl2 sdl2_image sdl2_mixer sdl2_ttf`. On Linux, install the SDL dev packages: `sudo apt install libsdl2-dev libsdl2-image-dev libsdl2-mixer-dev libsdl2-ttf-dev`. On Windows, `pip install pygame` bundles everything.

Step 2: The Architect (Plan Mode)

Start the agent. We begin in plan mode because we want a blueprint before we lay bricks.

```
1 python nanocode.py
```

The Prompt:

```
1 Build a classic Snake game using Pygame. Include a score counter and Game Over
  → screen with a restart option. Put ALL code in ONE file: snake.py. Write the
  → plan in PLAN.md.
```

The agent will use `write_plan` to create `PLAN.md`. Read it. It should outline the Snake class, the Food class, and the game loop—all in a single file.

If the plan looks reasonable, move on to Step 3.

Step 3: The Builder (Act Mode)

Switch to act mode:

```
1 /mode act
```

The Prompt:

```
1 Implement the plan in snake.py. All code in one file.
```

Watch the terminal:

```
1 → Writing snake.py
```

The agent is generating code based on the context it stored in `PLAN.md`.

Step 4: The Reality Check

Before running the game, bump the timeout. The default 30 seconds isn't enough to actually play a game—Pygame's main loop blocks until you close the window. Set the environment variable before launching:

```
1 export NANOCODE_TIMEOUT=300
```

The Prompt:

```
1 Run the game with: python snake.py
```

The agent executes `run_command`. A window pops up. You play Snake.

Dev Tip: The game window blocks the agent. When you run `python snake.py`, the agent waits until you close the game window before continuing. This is normal—Pygame’s main loop holds the terminal.

If it crashes: LLMs are non-deterministic. Your agent might produce a bug on the first try. If the game crashes with an error like `AttributeError: 'Snake' object has no attribute 'draw'`, don’t fix it yourself. Let the agent see the `stderr`.

The Prompt:

- 1 The game crashed. Read the error and fix it.

The agent will read the traceback, use `read_file` to find the bug, use `edit_file` to patch it, and run it again.

Step 5: The Pivot (Feature Creep)

The game works, but it’s ugly. The snake is just green squares. Let’s stress-test the agent’s ability to refactor.

The Prompt:

- 1 The game looks boring. Make the snake change color as it eats food, increase
→ speed every 5 points, and search the web for 'cool retro game color
→ palettes' to apply.

The agent should:

1. Use `search_web` to find color palettes
2. Use `read_file` to understand the current rendering logic
3. Use `edit_file` to inject the new features
4. Run the game to verify

What Goes Wrong

Your results will differ from mine—LLMs are non-deterministic. But here's what typically happens, and what to watch for.

Common failures on the first run:

- `ModuleNotFoundError`: No module named 'pygame' – the agent forgot you need to install it, or it ran the script in a different environment. Tell it to `run pip install pygame` first.
- `AttributeError` on a method the agent defined but misspelled in the call site. The agent fixes these quickly once it sees the traceback.
- Off-by-one errors in collision detection. The snake walks through walls or dies one pixel too early. These take 2-3 iterations of edit-run-fix.

The typical session arc:

In my tests, the agent usually gets a working (but ugly) game in 2-4 iterations. The first write produces something that crashes. The second or third fix gets it running. The feature creep step (color changes, speed ramps) adds another 3-5 iterations as the agent reads its own code, makes surgical edits, and verifies each change.

By the end, the conversation is 15-20 rounds deep. If you're using Claude, watch the compaction trigger—around round 12-15 you'll see "(Compacting conversation...)" as the token count approaches the 75% threshold. After compaction, the agent loses some detail about early rounds but keeps working. This is the system from Chapter 9 earning its keep.

Where the agent struggles:

Pygame's coordinate system and event loop are tricky. The agent sometimes writes code that renders correctly but doesn't handle keyboard input properly, or that draws the snake in the wrong order so the head appears behind the body. These are the kind of bugs a human spots instantly but the agent can't see—it has no visual feedback, only stdout and stderr. If the game runs without errors but looks wrong, you'll need to describe the visual bug: "The snake renders backwards—the head should be at the front."

The point isn't perfection on the first try. It's that the agent *converges*—write, run, read the error, fix, repeat—using every tool we built across eleven chapters.

Before you ship the final `snake.py`, read it. The agent writes code that *runs*, but a human should still audit for things the test loop can't catch: hardcoded magic numbers, missing edge cases (what if the window is resized?), and whether the code is structured in a way you'd want to maintain. The agent is a fast drafter, not a final reviewer.

Wrapping Up

Plan, implement, crash, debug, fix, run again—that's every chapter earning its keep.

So where does it go from here?

Epilogue

The whole thing is about 750 lines of Python. No framework. `nanocode.py` is yours. Do whatever you want with it:

- Git integration that auto-commits after green tests
- Screenshot-based debugging for frontend work (Claude can read images)
- Voice input via Whisper so you can talk instead of type
- MCP for connecting to external services your team already uses
- Sub-agents that fork conversations and work on subtasks in parallel

Production agents like Claude Code, Cursor, and Copilot do more than this: streaming responses (see Appendix A), parallel tool execution, tree-sitter parsing, sandboxed execution environments, multi-file context windows spanning thousands of files. The gap between 750 lines and 750,000 is real. But the architecture is the same: a brain, a loop, tools, memory, and a safety harness. Now you know what's behind the curtain.

The models will keep getting better. The harness—the loop, the tools, the safety checks—that part is engineering. And that part isn't going anywhere.

Appendix A: Streaming Responses

Throughout this book, we've used synchronous API calls—send a message, wait for the full response, then display it. This works fine for testing and keeps the code simple. But when Claude takes 15 seconds to think through a complex problem, staring at a blank terminal feels broken.

Streaming fixes this. Instead of waiting for the complete response, the API sends tokens as they're generated. The user sees text appear word by word, like watching someone type. Extended thinking streams too—the  reasoning appears in real time, so you know the agent is working before the response even starts.

How Streaming Works

Claude's Messages API supports streaming via Server-Sent Events (SSE). You set `"stream": true` in the payload, and instead of a single JSON response, you get a series of events:

```
1 event: content_block_start
2 data: {"type":"content_block_start","index":0,"content_block":{"type":"thinki
  ↳ ng","thinking":""}}
3
4 event: content_block_delta
5 data: {"type":"content_block_delta","index":0,"delta":{"type":"thinking_delta
  ↳ ","thinking":"Let
  ↳ me"}}
6
7 event: content_block_delta
8 data: {"type":"content_block_delta","index":0,"delta":{"type":"thinking_delta
  ↳ ","thinking":"
  ↳ analyze"}}
9
10 event: content_block_stop
11 data: {"type":"content_block_stop","index":0}
12
13 event: content_block_start
14 data: {"type":"content_block_start","index":1,"content_block":{"type":"text",
  ↳ "text":""}}
```

```

15
16 event: content_block_delta
17 data: {"type":"content_block_delta","index":1,"delta":{"type":"text_delta","t
    ↳ ext":"Here"}}
18
19 event: content_block_delta
20 data: {"type":"content_block_delta","index":1,"delta":{"type":"text_delta","t
    ↳ ext":" is the
    ↳ fix"}}
21
22 event: message_stop
23 data: {"type":"message_stop"}

```

Thinking blocks arrive first as `thinking_delta` events, followed by text as `text_delta` events. You print each immediately—thinking in dimmed text, response text in normal text.

The Implementation

We move all streaming logic into a shared `_stream_response` method on the `Brain` base class. Every brain—Claude, DeepSeek, Ollama—inherits it. Each subclass's `think()` builds its payload (with or without extended thinking) and delegates to `_stream_response`.

```

class Brain:
    """Base class for LLM providers."""
    context_limit = 200_000
    last_input_tokens = 0

    def think(self, conversation):
        raise NotImplementedError

    def _stream_response(self, url, headers, payload, max_retries=10):
        """Stream an API response, printing tokens as they arrive."""
        for attempt in range(max_retries):
            try:
                response = requests.post(url, headers=headers, json=payload,
                                         stream=True, timeout=120)
            except requests.exceptions.RequestException as e:
                wait_time = 2 ** attempt
                print(f"Network error: {e}. Retrying in {wait_time}s...")
                time.sleep(wait_time)
                continue

```

```

if response.status_code == 429 or response.status_code >= 500:
    retry_after = response.headers.get("retry-after")
    try:
        wait_time = int(retry_after) if retry_after else 2 ** attempt
    except (ValueError, TypeError):
        wait_time = 2 ** attempt
    print(f"Error {response.status_code}. Retrying in
    ↪ {wait_time}s...")
    time.sleep(wait_time)
    continue

if response.status_code >= 400:
    try:
        error_msg = response.json()["error"]["message"]
    except (KeyError, ValueError):
        error_msg = response.text
    raise Exception(f"API error ({response.status_code}):
    ↪ {error_msg}")

break
else:
    raise Exception(f"Request failed after {max_retries} retries")

blocks = {} # Per-block state by index
block_parts = {} # Per-block text accumulator
# ... tool_calls, current_tool, current_index, input_tokens ...

for line in response.iter_lines():
    # ... skip empty lines, decode, extract "data: " prefix ...

    data = json.loads(line[6:])
    event_type = data.get("type")

    if event_type == "message_start":
        input_tokens = ...

    elif event_type == "error":
        raise Exception(f"Stream error: {data.get('error',
    ↪ {}).get('message', data)}")

    elif event_type == "content_block_start":
        block = data.get("content_block", {})
        current_index = data.get("index", 0)
        blocks[current_index] = {"type": block.get("type")}
        block_parts[current_index] = []
        # ... init tool_use or print 🗨 for thinking ...

    elif event_type == "content_block_delta":
        delta = data.get("delta", {})
        if delta.get("type") == "thinking_delta":

```

```

        print(delta["thinking"], end="", flush=True)
        block_parts[current_index].append(delta["thinking"])
    elif delta.get("type") == "text_delta":
        print(delta["text"], end="", flush=True)
        block_parts[current_index].append(delta["text"])
    # ... input_json_delta for tool args ...

    elif event_type == "content_block_stop":
        # Finalize: join block_parts[idx], store in
        ↪ blocks[idx]["content"]
        # Reset dim for thinking, parse JSON for tool_use

    # Build raw_content in index order (matches API's content array)
    raw_content = []
    for idx in sorted(blocks.keys()):
        # Emit each block's own content in original order

    return Thought(text=full_text, tool_calls=tool_calls,
                   raw_content=raw_content, thinking=full_thinking)

```

Each brain's `think()` becomes a thin wrapper—build the payload, call `_stream_response`:

```

class Claude(Brain):
    def think(self, conversation):
        headers = { ... }
        payload = {
            "model": self.model,
            "max_tokens": 16000,
            "thinking": {"type": "enabled", "budget_tokens": 10000},
            "messages": conversation,
            "stream": True,
        }
        # ... memory, tools ...
        return self._stream_response(self.url, headers, payload)

```

DeepSeek and Ollama use the same pattern without the thinking parameter – not because their APIs lack support (both support it), but to keep the code simple. Adding thinking to DeepSeek or Ollama is a one-line payload change; the SSE parsing already handles it.

The Walkthrough:

The parser is the most stateful code in the book, so it's worth tracing the key decisions:

- **blocks and block_parts** track each content block by its index from the SSE events. The API assigns sequential indices (0, 1, 2...) to content blocks. Each block gets its own text accumulator, so multiple text blocks or thinking blocks don't bleed into each other. At the end, we reconstruct `raw_content` in index order – critical for conversation history, since Claude expects the exact structure back.
- **ANSI dim mode** – when a thinking block starts (`content_block_start` with type "thinking"), we print `\033[2m` (dim) and the `🤖` prefix. Every subsequent `thinking_delta` prints dimmed. When `content_block_stop` fires for a thinking block, `\033[0m` resets to normal – and the text that follows renders normally.
- **current_tool** accumulates partial JSON for tool calls. Tool arguments arrive as `input_json_delta` fragments that must be concatenated before parsing. When the tool's `content_block_stop` arrives, we parse the complete JSON string and create a `ToolCall`.
- **Error events** raise immediately. Anthropic's streaming can emit error events (e.g., overloaded servers) mid-stream. Without this check, the method would return an empty `Thought` and the agent would silently proceed with no response.

What Changed from Chapter 11

- **`_parse_response` and `request_with_retry` are gone.** Both are replaced by `_stream_response`, which handles streaming, retry logic, and response parsing in one method. No more waiting for the full response before displaying anything.
- **"stream": True** in every brain's payload tells the API to send events.
- **`stream=True`** in `requests.post()` tells the requests library to yield data as it arrives.
- **`response.iter_lines()`** reads one SSE event at a time.
- **`print(text, end="", flush=True)`** prints each delta immediately, without a newline, flushing the buffer so it appears instantly.
- **Extended thinking streams in real time** – `thinking_delta` events arrive before `text_delta` events. The `🤖` prefix prints once when the thinking block starts, then thinking tokens stream dimmed. When the block ends, we reset the ANSI dim code.

- **Tool calls arrive in pieces** – the tool name comes in `content_block_start`, and the JSON input streams through `input_json_delta` events. We accumulate the partial JSON and parse it when the block ends.
- **Token tracking** comes from the `message_start` event instead of the final response body.
- **SSE error events** are caught and raised as exceptions. Without this, a mid-stream server error would silently produce an empty response.
- **raw_content preserves block order** by tracking each block's index from the SSE events and emitting them in order at the end.
- **The agentic loop simplifies** – since `_stream_response` prints text and thinking inline, the loop no longer needs to display thinking after the fact. The loop returns "" (text was already streamed), so `main()` only prints non-empty return values from commands like `/mode` and `/switch`.

The Trade-Off

Streaming adds complexity. The synchronous path from Chapter 11 (`think()` + `_parse_response` + `request_with_retry`) is about 50 lines across three methods; `_stream_response` is 80+ lines in one method. The event parsing is fiddly—partial JSON accumulation, multiple event types, state tracking for tools and thinking blocks.

For the teaching chapters, that's why we kept it synchronous. For a tool you use daily, streaming is worth the cost. The agent feels faster even when it isn't, because you can read the first sentence while it's still generating the last. And thinking feedback is the biggest win—without streaming, you'd stare at a frozen prompt for 5-10 seconds during reasoning. With it, you see the model working in real time.

Running the Code

The complete runnable code is in `resources/code/appendix/nanocode.py`, with tests in `test_nanocode.py`. The tests verify SSE event parsing and Thought assembly in isolation—they exercise the parsing logic (event routing, thinking accumulation, tool JSON reassembly, error handling) without hitting the real API or testing `_stream_response` directly.



Aside: All three brains share `_stream_response` because they all use the Anthropic-compatible SSE format. That's the Adapter Pattern earning its keep one more time—the same streaming parser handles Claude, DeepSeek, and Ollama.

Acknowledgments

To my wife: Through late nights and the moments I doubted this would ever become a book—you believed in me. You kept everything running so I could chase this. I love you, and I couldn't have shipped it without you.

To my son: Every time you ask “why?” or take a toy apart to see how it works, you remind me why I wrote this book. I hope one day you'll read these pages and know your dad was thinking of you with every word.

To my parents: You raised me to ignore the hype and understand the raw materials. The philosophy behind this book—skip the magic, do the work yourself—is something I learned from watching you. Thank you for sacrificing so much to give me the tools to build my own path.

To my grandpa and grandma: I miss you. You left before you could see this, but you're the reason it exists. This book is for your memory.